

National Technical University of Athens

School of Electrical and Computer Engineering

Division of Computer Science

OS mechanisms for efficient memory management and filesystems for serverless functions in virtual machines

INTERMEDIATE THESIS

ORESTIS LAGKAS NIKOLOS

Athens, March 2025



National Technical University of Athens
School of Electrical and Computer Engineering
Division of Computer Science

OS mechanisms for efficient memory management and filesystems for serverless functions in virtual machines

INTERMEDIATE THESIS

ORESTIS LAGKAS NIKOLOS

Advisors:

Nectarios Koziris
Nikolaos Papaspyrou
Georgios Goumas

. . . . . .
Nectarios Koziris
Professor
National Technical University of Athens

. . . . . .
Nikolaos Papaspyrou
Professor
National Technical University of Athens

. . . . . .
Georgios Goumas
Professor
National Technical University of Athens

. . . . . .
Dionisios N. Pnevmatikatos
Professor
National Technical University of Athens

. . . . . .
Stergios V. Anastasiadis
Professor
University of Ioannina

Athens, March 2025

. . . . . .

Orestis Lagkas Nikolos

Copyright © Orestis Lagkas Nikolos, 2025
Με επιφύλαξη παντός δικαιώματος. All rights reserved.

Απαγορεύεται η αντιγραφή, αποθήκευση και διανομή της παρούσας εργασίας, εξ ολοκλήρου ή τμήματος αυτής, για εμπορικό σκοπό. Επιτρέπεται η ανατύπωση, αποθήκευση και διανομή για σκοπό μη κερδοσκοπικό, εκπαιδευτικής ή ερευνητικής φύσης, υπό την προϋπόθεση να αναφέρεται η πηγή προέλευσης και να διατηρείται το παρόν μήνυμα. Ερωτήματα που αφορούν τη χρήση της εργασίας για κερδοσκοπικό σκοπό πρέπει να απευθύνονται προς τον συγγραφέα.

Οι απόψεις και τα συμπεράσματα που περιέχονται σε αυτό το έγγραφο εκφράζουν τον συγγραφέα και δεν πρέπει να ερμηνευθεί ότι αντιπροσωπεύουν τις επίσημες θέσεις του Εθνικού Μετσόβιου Πολυτεχνείου.

Contents

1 Introduction **17**

1.1 Problem Statement . . . 17

1.2 Thesis Approach and Contributions . . . 18

2 Background **21**

2.1 Serverless . . . 21

2.2 Serverless sandboxing . . . 22

2.2.1 Common Sandboxing terms . . . 22

2.2.1.1 Container image and layers . . . 22

2.2.1.2 Container . . . 22

2.2.2 Sandboxing isolation . . . 22

2.2.2.1 OS-level virtualization: Containers . . . 22

2.2.2.2 Hypervisor-based virtualization . . . 23

2.2.3 Sandboxing runtimes . . . 23

2.2.3.1 High level container runtimes . . . 23

2.2.3.2 Low level runtimes . . . 23

2.2.4 Container storage mechanisms . . . 25

2.2.4.1 File based storage . . . 26

2.2.4.2 Block based storage . . . 26

2.2.5 VM storage mechanisms . . . 26

2.2.5.1 VM Block based storage . . . 26

2.2.5.2 VM File based storage . . . 27

2.2.6 Container storage drivers and VM compatibility . . . 27
2.3 Serverless isolation models . . . 28
2.3.1 Single function per VM . . . 28
2.3.2 Multiple functions per VM . . . 28
2.3.3 Trade-offs . . . 29
2.3.3.1 Cold starts . . . 29
2.3.3.2 Resource sharing . . . 29
2.3.3.3 Resource allocation . . . 29
2.4 VM Memory Elasticity . . . 30
2.4.1 Memory hotplugging . . . 30
2.4.2 Memory initialization . . . 31
2.4.3 Physical Memory Allocation . . . 31
2.4.4 CPU interference . . . 33

3 Storage for functions in VMs 35
3.1 Motivation . . . 37
3.2 Container storage for VMs . . . 39
3.2.1 Overview . . . 40
3.2.2 Layer and Container Preparation . . . 40
3.2.3 Volume provisioning . . . 40
3.2.4 Filesystem Mount . . . 42
3.3 Evaluation . . . 44
3.3.1 Methodology . . . 44
3.3.2 Boot Time . . . 45
3.3.2.1 Single Instance scenario . . . 45
3.3.2.2 Multiple Instances scenario . . . 47
3.3.2.3 Scaling Concurrently scenario . . . 48
3.3.3 Memory footprint . . . 49
3.3.4 I/O Redundancy . . . 50
3.3.5 I/O Performance . . . 51
3.3.6 Image Pull . . . 52
3.4 Discussion . . . 53
3.4.1 Asynchronous Pool management . . . 53
3.4.2 Bind mount or device mount . . . 53
3.5 Container storage for Unikernels . . . 54
3.5.1 Enabling container storage for Unikernel Containers . . . 54
3.5.2 Storage classes for Unikernel Containers . . . 54

3.5.3 Container storage for unikernels . . . 55
3.5.3.1 Extending the container image building process . . . 55
3.5.3.2 Extend Nabla containers . . . 56
3.5.4 Experimental results . . . 58

4 Scaling functions in VMs with memory elasticity 61
4.1 Motivation . . . 62
4.2 Memory Reclamation for Serverless functions . . . 63
4.2.1 Overview . . . 63
4.2.2 Memory Partitioning . . . 63
4.2.3 Distinguishing shared and private allocations . . . 64
4.2.4 Partition aware unplugging . . . 64
4.2.5 Implementation details . . . 64
4.2.6 Integration with a FaaS Runtime . . . 67
4.3 Evaluation . . . 69
4.3.1 Methodology . . . 69
4.3.2 Microbenchmarks . . . 71
4.3.2.1 Unplug Speed . . . 71
4.3.2.2 CPU Usage during unplug . . . 72
4.3.3 Serverless workloads with real traces . . . 74
4.3.3.1 Memory Reclamation . . . 74
4.3.3.2 Interference during unplug . . . 74

5 Concluding the Thesis 77
5.1 VM Memory Elasticity . . . 77
5.1.1 Reclamation chunk size . . . 77
5.1.2 Evaluation . . . 77
5.1.2.1 End to end experiment . . . 78
5.1.2.2 Comparison with VM ballooning and related work . . . 78
5.1.2.3 Comparison with single instance model . . . 78
5.1.2.4 Memory utilization . . . 78
5.1.2.5 Memory zeroing overheads . . . 78
5.2 Future Work . . . 79
5.2.1 Scaling Functions with VM snapshots . . . 79
5.2.1.1 Proposal Design Overview . . . 81
5.2.2 Multiple Guests page cache deduplication . . . 82
5.2.3 Optimizing storage for functions in VMs by identifying I/O patterns . . . 83

6 Related Work **85**
6.1 Memory Elasticity 85
6.2 Serverless Scaling 86
6.3 Resource Harvesting 86
6.4 Container storage 87

7 Summary **89**

List of Figures

2.1 Container runtimes with different sandboxing and storage mechanisms 24
2.2 The 1:1 and N:1 serverless models. 28
2.3 Processes footprints interleaved within a single physical memory block. 32

3.1 Startup latency for multiple host containers from the same image 37
3.2 Container preparation with the *DeverlayMount* struct 41
3.3 Overview of *Provisioner* design and workflow 41
3.4 Single instance startup time for VM container. 46
3.5 Single instance startup time for host container. 46
3.6 Sequential startup of 128 nginx instances. 47
3.7 scaling with parallel container spawning. 48
3.8 scaling with parallel container spawning for different batch sizes. 48
3.9 Total cache usage for different container storage drivers. 50
3.10 Data accesses on the physical disk for different container storage drivers. 50
3.11 64KB sequential read throughput by parallel VM containers from the same image. 51
3.12 Build process of a container unikernel image 55
3.13 Create process of a container unikernel using filesystem images as individual storage layers. 57
3.14 Time needed to generate the `rootfs.iso` as a function of the size of the container image 58
3.15 Storage space used as a function of the number of containers running on the system 59

4.1 ZONE_SQUEEZY partitions and memory allocation 63
4.2 Squeezy integration to a serverless runtime 67
4.3 Unplug VM memory performance. 71
4.4 Cumulative CPU utilization for unpluging 73
4.5 Throughput (MiB/sec) of reclaiming function instances memory based on a realistic FaaS trace experiment. 73
4.6 P99 latency of functions invocations. 73
4.7 Average per second invocation end-to-end latency during memory shrinking events 75

5.1 Comparison of the 1:1 and N:1 optimized models. 80
5.2 High level overview of the proposed hybrid model. 81

List of Tables

3.1 Deverlay Components 35
3.2 Deverlay Evaluation System Configuration. 44
3.3 Deverlay Evaluation Workloads 44
3.4 Time to pull an image for different container storage drivers. 52

4.1 Squeezy evaluation workloads. 69

Περίληψη

Το μοντέλο εκτέλεσης Συναρτήσεων ως Υπηρεσία (ΣωΥ) έχει κυριαρχίσει στον τομέα του Υπολογιστικού Νέφους (Cloud Computing) [1–4, 11, 13]. Οι εφαρμογές ΣωΥ αναπτύσσονται ως αυτόνομες συναρτήσεις για τις οποίες οι χρήστες παρέχουν τον πηγαίο κώδικα ενω οι πάροχοι διασφαλίζουν τις εξαρτήσεις (π.χ. σύστημα εκτέλεσης) και τους απαραίτητους πόρους (μνήμη, υπολογιστική ισχύς κλπ) για την εκτέλεσή τους. Με αυτόν τον τρόπο απλοποιείται ο προγραμματισμός - οι χρήστες απαλλάσονται απο τη διαχείριση ενός ολόκληρου συστήματος - και η διαχείριση των πόρων - ο πάροχος μπορεί να κλιμακώσει δυναμικά και με διαφάνεια τους πόρους που απαιτώνται από κάθε συνάρτηση με βάση τον ερχόμενο φόρτο αιτημάτων. Συνήθως, η κατανομή πόρων ελέγχεται με αύξηση ή μείωση του αριθμό των στιγμιοτύπων των συναρτήσεων που εξυπηρετούν αιτήματα.

Αυτή η διατριβή εστιάζει στους μηχανισμούς του Λειτουργικού Συστήματος (ΛΣ) που χρησιμοποιούνται για τα συστήματα αρχείων των στιγμιοτύπων των συναρτήσεων καθώς και για τη διαχείριση της μνήμης που χρησιμοποιούν. Αρχικά εξέτασα τα συστήματα χρόνου εκτέλεσης και απομόνωσης που χρησιμοποιούνται για εφαρμογές χωρίς διακομιστή και εντόπισα τα κόστη που απαιτούν οι πρώτες εκκινήσεις Συναρτήσεων (cold starts) και οφείλονται στο σύνορο που εισάγει η εικονικοποίηση: το σύστημα οικοδεσπότη και το εικονικό σύστημα διατηρούν διαφορετικές σημασιολογίες στα συστήματα αρχείων τους. Για το σκοπό αυτό σχεδίασα και υλοποίησα έναν οδηγό αποθήκευσης για συναρτήσεις που εκτελούνται ως δοχεία (containers) μέσα σε εικονικές μηχανές. Η πρότασή μου γεφυρώνει το χάσμα μεταξύ των δοχείων που εκτελούνται σε εικονικές μηχανές και των παραδοσιακών δοχείων που εκτελούνται στον οικοδεσπότη, καθώς μοιράζονται αποτελεσματικά τις εξαρτήσεις τους αξιοποιώντας την προσωρινή μνήμη κεντρικών σελίδων του οικοδεσπότη. Στη συνέχεια, μετά από διερεύνηση πιθανών λύσεων που στοχεύουν στην αύξηση της δυνατότητα

κοινής χρήσης μεταξύ στιγμιοτύπων συναρτήσεων με στόχο τη μείωση των χρονοκαθυστερήσεων των αιτημάτων και την αποτελεσματική χρήση πόρων, βελτίωσα το μοντέλο εκτέλεσης πολλαπλών δοχείων σε μία εικονική μηχανή εισάγοντας τη δυνατότητα για αποτελεσματική ελαστικότητα στην μνήμης της εικονικής μηχανής. Σε αυτό το μοντέλο πολλαπλά στιγμιότυπα συναρτήσεων μοιράζονται την ίδια Εικονική Μηχανή και επομένως χωρίς τη δυνατότητα ελαστικής διαχείρισης της μνήμης, οι περιττές δεσμεύσεις μνήμης περιορίζουν την απόδοση του μοντέλου. Η προτεινόμενη λύση μου συγκεντρώνει και τακτοποιεί τις δεσμεύσεις μνήμης των συναρτήσεων σε διαφορετικά διαμερίσματα μνήμης μέσα στην εικονική μηχανή, διευκολύνοντάς την ταχεία αποδέσμευση αυτής της μνήμης κατά τον τερματισμό τους. Για να ολοκληρώσω τη διδακτορική μου διατριβή, σκοπεύω να βελτιστοποιήσω περαιτέρω το μηχανισμό (απο)δέσμευσης της μνήμης των εικονικών μηχανών προσθέτοντας τη δυνατότητα να ανακτάται η μνήμη σε μεγαλύτερα μεγέθη και επομένως να εξαλείφει πλήρως η καθυστέρηση ανάκτησης σε σύγκριση με πρόσφατες σχετικές τεχνολογίες. Σκοπεύω επίσης να συμπεριλάβω στην ενότητα αξιολόγησης του μηχανισμού την εκτέλεση ενός πιο γενικευμένου πειράματος που σκοπό θα έχει να αναδείξει την αποδοτικότητα διαχείρισης της μνήμης του συστήματος και τον αντίκτυπο της στη δημιουργία νέων στιγμιοτύπων συναρτήσεων. Επιπλέον, σκοπεύω να συμπεριλάβω μία σύγκριση με άλλους μηχανισμούς ελαστικότητας μνήμης, π.χ. αερόστατο (balloon) καθώς και μία αναλυτική καταμέτρηση των επιμέρους επιβαρύνσεων στο συνολικό κόστος αποδέσμευσης της μνήμης, π.χ. επιβάρυνση για μετεγκατάσταση / αντιγραφή χρησιμοποιούμενων σελίδων, επιβάρυνση μηδενισμού της υπο αποδέσμευση μνήμης κλπ. Τέλος, σχεδιάζω να συμπεριλάβω μια σύγκριση των επιμέρους επιβαρύνσεων στις χρονοκαθυστερήσεις ουράς και της χρήση της μνήμης σε αντιδιαστολή με το μοντέλο ενός δοχείου ανά εικονική μηχανη (single-container-per-VM) όπου κάθε νέα μοναδικό στιγμιότυπο τοποθετείται σε ένα μοναδικό microVM.

Μελλοντικά, η απόδοση χωρίς διακομιστή θα μπορούσε επίσης να διερευνηθεί περαιτέρω στην τελευταίας τεχνολογίας κατεύθυνση της χρήσης στιγμιότυπων μνήμης εικονικών μηχανών:

- εξερεύνηση του μοντέλου πολλών δοχείων ανά εικονική μηχανή σε συνδυασμό με τη χρήση στιγμιότυπων μνήμης εικονικών μηχανών που έχουν σχεδιαστεί να φιλοξενουν πολλαπλά στιγμιότυπα συναρτήσεων.

- εξερεύνηση ενός μηχανισμού στιγμιότυπων μνήμης εικονικών μηχανών διατηρούμενων στη μνήμη. Ένας τέτοιος μηχανισμός μπορεί να κλονοποιήσει δυναμικά μία εικονική μηχανή που φιλοξενεί πολλαπλά στιγμιότυπα συναρτήσεων χωρίς να χρειάζεται πρώτα να αντιγράψει σειριακά την μνήμη στο δίσκο - εξάλειψη προσβάσεων εισόδου/εξόδου (ΕΕ) κατά το χρόνου κλονοποίησης.

Abstract

The Function-as-a-Service (FaaS) execution model has dominated cloud computing [1–4, 11, 13]. FaaS applications are developed as standalone functions for which users provide the source code, while providers secure the dependencies (e.g., runtime) and the necessary resources (e.g., memory, CPU) for their execution. This simplifies programming, i.e. developers are alleviated from the burden to manage an entire system, and enables better resource management, i.e. providers can scale dynamically and transparently the resources allocated per function based on the incoming load of requests. Typically, resource allocation is controlled by increasing or decreasing the number of function instances that serve requests.

This thesis focuses on the OS mechanisms used for function instances filesystems and for function memory management. I initially surveyed the sandboxing runtimes used in serverless and identified the overhead introduced in the functions cold starts caused by the virtualization barrier: host and guest have different filesystem semantics. To that end, I designed and implemented a storage driver for functions executed as containers inside Virtual Machines. My proposal (published in [78,79]) narrows the gap between guest containers and traditional containers as it efficiently shares function dependencies by leveraging host page cache. Furthermore, after exploring solutions that increase the shareability between instances targeting better latencies and efficient resource usage, I optimized the multi-container-per-VM model by introducing efficient memory elasticity for Virtual Machines. In this model multiple instances share the same Virtual Machine and thus without memory elasticity redundant allocations limit the model's efficiency. My proposed solution, aggregates function allocations to different memory partitions inside the guest, facilitating their rapid reclamation upon eviction. To conclude my PhD dissertation, I intend to further optimize the VM memory unpluging mechanism in order to reclaim

memory in larger chunks and thus completely eliminate the reclamation latency compared to state-of-the-art. I also intend to enhance the evaluation section of this mechanism to include end-to-end experiment showing the system memory efficiency and the impact on spawing new instances. Moreover, I plan to include comparison with other memory elasticity mechanisms, e.g. ballooning as well as a breakdown of (un)plugging overheads, e.g. page migrations overhead, memory zeroing overhead. Finally I plan to include a comparison with the single-container-per-VM model where each new instance is placed in a fresh microVM, and measure the tail latencies breakdown and the memory usage in each scenario.

Finally, serverless performance could be further explored in the future, focusing on the recent state-of-the-art of using snapshots:

- exploring the multi-container-per-VM model when using microVM snapshots designed to host multiple function instances.
- exploring an in-memory snapshot mechanism that can dynamically fork an running microVM hosting multiple functions without serializing its state to disk - eliminate I/O from the runtime path.

CHAPTER 1

Introduction

1.1 Problem Statement

Serverless computing, in the form of the Function-as-a-Service (FaaS) execution model, has in recent years solidified its status as the dominant cloud computing paradigm studied by both industry and academia [1–4, 11, 13]. FaaS applications are developed as standalone functions, designed to be highly available. Users typically provide their source code and providers are responsible to secure the dependencies (e.g., runtime) and the resources (e.g., memory, CPU) for the functions to run. One of the defining features of FaaS is its pay-as-you-go model, in which providers execute functions on-demand and charge users only for the resources consumed during function execution. This critical feature requires up- and down-scaling compute and memory resources dynamically and transparently following the incoming load.

Typically, providers scale the resources by increasing or decreasing the number of function instances that serve requests. A new function instance is created when there are no other available instances to handle the incoming request(s) within the typical Service Level Objective (SLO) agreement (e.g., tail latency). A function instance is evicted when the incoming load falls below a threshold. However, in order to reach the required SLOs with high resource efficiency, the increase and decrease of the number of functions according to the incoming load needs to be paired with an appropriate provisioning and de-provisioning of actual resources.

Functions are commonly sandboxed within either containers, e.g., Docker [7], or (micro) virtual machines (microVMs), e.g., Firecracker [30]. FaaS providers typically eschew containerization for the strong isolation guarantees of microVMs. When deploying functions in VMs, FaaS providers opt for different isolation models, namely: a) the *single-container-per-VM* model, i.e. each function instance is deployed in a dedicated microVM (AWS Lambda [2]), or b) the *multi-container-per-VM* model, i.e. multiple instances or different functions of the same tenant share the same microVM (Azure [74]).

One important observation is that both models require the creation and execution of an "infrastructure" illusion - sandbox - which serves as the function host. Typically a portion of the system software - runtime - involved in the execution is similar or even identical among function instances. FaaS introduces a shareability potential coming from the native duplication of dependencies for multiple functions. On the other hand, sandboxing is contradictory, as deduplication is incommoded by the isolation gap inserted among function instances. The above strategies offer different trade-offs in terms of shareability, performance and resource allocation.

Serving an incoming request from a freshly-spawned function instance (*cold start*) severely impacts tail latency compared to using a cached (*warm*) function instance [86, 92]. Cold-start delays, though, vary significantly based on the isolation policy. Functions on dedicated microVMs suffer from higher start-up delays as cold runs include booting the Guest OS and initializing the function's runtime after unpacking the function code. Moreover, deduplication is undermined by the virtualization gap mainly by maintaining multiple page caches in different guest's address spaces. Instead, sharing the VM decouples new instances from such initialization costs and reduces the memory tax per instance as they typically share some of their runtime dependencies (e.g., libraries). However, this method requires overprovisioned VMs (e.g., in terms of CPU and / or memory resources) to be able to scale up the number of instances within them when the incoming load increases. This has a significant impact on resource utilization, as the VMs tie down idle resources, especially when the incoming load is low. At the same time, larger VMs are harder to place and schedule across the cluster and can lead to resource fragmentation [42, 43, 64, 73].

1.2 Thesis Approach and Contributions

This thesis argues that we need to rethink how the traditional OS mechanisms and concepts, targeting general purposed systems with shared address spaces, are now used for Serverless systems relying on multiple - guest - address spaces. The goal of this thesis is to narrow the gap between the isolation abstraction, i.e. hardware virtualization, and the physical hardware used for Serverless workloads, while optimizing performance metrics such as initialization overheads and memory usage.

To achieve this goal, the OS need to provide efficient mechanisms to runtimes, that leverage the shareability potential of Serverless model. Identical bits in the software stacks and dependencies of function instances execution need to be de-duplicated among multiple Virtual Machines sandboxes. Moreover the physical resources asked for workloads should be rapidly allocated or reclaimed on demand.

This thesis proposes two mechanisms targeting the above demands: Deverlay and Squeezy. Deverlay maximizes the shareability of function's software dependencies among different microVMs by implementing a novel storage driver for containers executed in Virtual Machines. Squeezy introduces memory elasticity in the guest OS and implements an efficient mechanism for (re)claiming chunks of physical memory for / from function instances in microVMs.

The first part of the thesis (ch. 2) provides a background discussion on sandboxing techniques used in Serverless and the OS mechanisms involved in different isolation abstractions, i.e. containers and Virtual Machines. It also focuses on the sandboxing runtimes proposed for deploying functions packaged in container images. Background paragraphs § 2.2.4 and § 2.2.5 specifically focus on storage mechanisms involved in preparing the sandbox and function execution, providing the required context for ch. 3 which describes the Deverlay proposal. The last two Sections provide the context for ch. 4 which describes the Squeezy proposal. § 2.3 describes in detail the two different isolation models in Serverless, i.e. *single-container-per-VM* and *multi-container-per-VM*, and their trade-offs in terms of initialization time and resource efficiency. § 2.4 focuses on memory elasticity for Virtual Machines by hot(un)plugging memory blocks and identifies the reasons behind the existing overheads on the OS memory management principles.

Following the background chapter is a detailed description of the design and implementation of Deverlay, concluding with the experimental evaluation of the proposal. Existing container storage drivers, i.e Devicemapper, is used to motivate Deverlay, by showing the existing inefficiency in the context of creating and spawning containers in VMs. The OS mechanisms involved in the creation path of virtualized containers introduce a large amount of I/O redundant operations while also lack support for sharing common dependencies among multiple instances on the same node. The above are identified as the main bottleneck in fast scaling scenarios when multiple function instances are concurrently scheduled on the server. The thesis proposal on container storage for VMs starts by introducing the idea of mapping the traditional container layers to separate block volumes plugged to multiple VMs. This design segregates the function dependencies and thus maximizes their shareability among multiple instances. The provisioner component, described next, maintains a pool of proctively provisioned virtual volumes and shifts the I/O operations related to their creation outside the initialization critical path. The design chapter concludes with the filesystem mounting operation which guarantees a union view of the segregated design inside the VM. Finally, the experimental evaluation results are presented, which show that Deverlay overcomes the virtualization gap and presents similar scaling behavior

with traditional host containers, by eliminating duplication and I/O redundancy. The discussion session proposes further optimizations targeted to faster filesystem preparation, as future work.

The thesis proposal on memory elasticity for Serverless is described in detail in chapter 4. In favor of the shareability benefits, the proposal embraces the multi-container-per-VM model where multiple functions share the same guest address space. The rigid resource allocation of this model, described in background, drives the motivation for designing a solution to dynamically and efficiently resize VM resources. While compute resources can be seamlessly adjusted on the fly [34, 101], memory elasticity for VMs remains challenging, especially when scaling memory down. The state-of-practice interfaces, namely memory ballooning [85, 91] and vDIMM memory hotplugging [68], suffer from unreliable or unpredictably slow memory reclamation, impact guest performance, and can induce fragmentation on both host and guest [10, 50, 69]. virtio-mem [50] achieves a middle ground between the two mechanisms and improves upon both. However, virtio-mem is not designed to provide the rapid memory elasticity required by the FaaS execution model, since, as shown in §4.3.3.1, it requires multiple seconds to resize a VM after 10s of function instances are recycled (scaled down). The bursty access pattern of serverless functions, however, frequently requires up and down scaling resources in sub-second intervals [54]. Providing an optimized hotplugging interface is only part of the solution. Processes that run in a VM (e.g., function instances) tend to have interleaved footprints due to the lazy memory allocation employed by modern OS. When a process exits, the released (idle) memory is fragmented, requiring the migration of interleaved occupied pages to be reclaimed. These migrations dominate the performance of VM down-sizing [69]. *We argue that memory hotplugging interfaces need to be coupled with informed memory management in the guest to ensure fast and reliable memory reclamation.*

CHAPTER 2

Background

2.1 Serverless

Serverless computing, in the form of the Function-as-a-Service (FaaS) execution model, has in recent years solidified its status as the dominant cloud computing paradigm studied by both industry and academia [1–4, 11, 13]. FaaS applications are developed as standalone functions, designed to be highly available. Users typically provide their source code and providers are responsible to secure the dependencies (e.g., runtime) and the resources (e.g., memory, CPU) for the functions to run. One of the defining features of FaaS is its pay-as-you-go model, in which providers execute functions on-demand and charge users only for the resources consumed during function execution. This critical feature requires up- and down-scaling compute and memory resources dynamically and transparently following the incoming load.

Typically, providers scale the resources by increasing or decreasing the number of function instances that serve requests. A new function instance is created when there are no other available instances to handle the incoming request(s) within the typical Service Level Objective (SLO) agreement (e.g., tail latency). A function instance is evicted when the incoming load falls below a threshold. However, in order to reach the required SLOs with high resource efficiency, the increase and decrease of the number of functions according to the incoming load needs to be paired with an appropriate provisioning and de-provisioning of actual resources.

2.2 Serverless sandboxing

In order to provide inter-tenant isolation, FaaS providers sandbox functions running in public clouds in isolated environments. A common practice is to ship and run function code and dependencies as a container [7]. However, containers based on OS-level virtualization (§ 2.2.2.1) do not provide strong security and isolation guarantees [72]. Consequently, FaaS providers typically resort to hardware level isolation, provided by hypervisor-based virtualization (§ 2.2.2.2), such as virtual machines (VMs) [30], for function sandboxing. Containers encapsulating function code and dependencies - in container images - are then run inside a VM. In the following subsection we explain how a container can be created and run from an image in various sandboxing scenarios involving different isolation mechanisms.

2.2.1 Common Sandboxing terms

2.2.1.1 Container image and layers

A container `image` is a stack of layers, where a `layer` is a tarball archive of the differences from its parent layer on the stack. The OCI Image Specification defines how to create an OCI Image bundle containing the stack of the filesystem layers, a manifest holding information about the stack and an initial configuration such as default arguments and variables. Each layer is unique but it can be shared between different images derived from it.

2.2.1.2 Container

A `container` is the result of the union of an empty Read Write (RW) filesystem layer on top of an OCI image.

2.2.2 Sandboxing isolation

2.2.2.1 OS-level virtualization: Containers

OS-level sanboxing relies the host OS mechanisms in order to create an illussion of a sandbox between host processes. Traditioanl containers in Linux are ”isolated” by using the mount, network and pid namespaces. Their resources are limited and controlled by the cgroups subsystems. Although the above mechanisms are lightweight and can be initialized faster, compared to a traditional Virtual Machine, the isolation is looser as all the processes share the same OS, kernel and address space.

2.2.2.2 Hypervisor-based virtualization

VM sandboxing offers strict isolation enforced in the hardware level. In contrast to traditional containers, the untrusted workload lives inside the guest address space and therefore the attack surface is significantly reduced. However, setting up a new Virtual Machine invokes an initialization overhead and therefore increases the boot time compared to regular containers. Firecracker microVMs [30] emerged from the need of quickly spawning lightweight Linux images with small memory footprint compared to QEMU. While both approaches are based on KVM and CPU virtualization extensions, Firecracker significantly improves boot times and memory overhead, compared to QEMU, by stripping down the VM configuration to the mininal subset of devices and kernel configuration options needed for Serverless workloads. However, compared to the OS-level virtualization provided by host containers, the boot time and the memory overhead of microVMs is still higher.

In an effort to ensure both performance and isolation, prior works have proposed the use of unikernels [59,61,70,81]: single address space images, packaged with only the absolute necessary components for the specific function to be executed. Unikernels are binary images and are executed as VMs on top of traditional VMMs (such as QEMU/KVM) or lightweight ones (Solo5 [95], Firecracker [30] etc.). Relevant research [52, 59, 70, 94] has shown the merits of unikernels for the execution of short-lived applications, mainly due to their minimal footprint and their near-instant spawn times. Due to their design, unikernels handle I/O (network and storage) through virtual devices: similarly to VMs, unikernels need a virtual network interface or a virtual block device in order to communicate with the network or access data from the host filesystem (§ 2.2.5). However most unikernel frameworks are still immature and shift the burden of system programming to the application developers.

2.2.3 Sandboxing runtimes

2.2.3.1 High level container runtimes

High level runtimes implement methods to unpack a downloaded image and prepare a run-able bundle, according to the OCI Spec. They implement different storage drivers (2.2.4) based on various OS mechanisms in order to apply the layers described in the image manifest. Docker and containerd are well known and widely used high level runtimes.

2.2.3.2 Low level runtimes

Lower level runtimes execute a runtime bundled, prepared by a higher level runtime, by mounting the filesystem and implementing different sandboxing techniques depending on what they

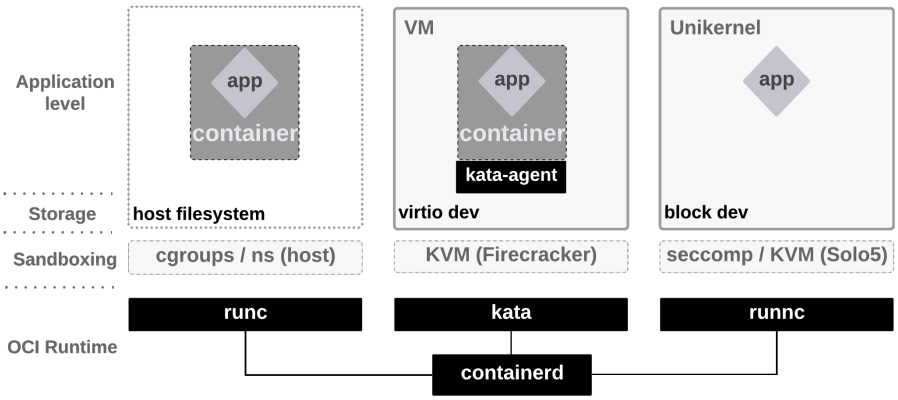


Figure 2.1: Container runtimes with different sandboxing and storage mechanisms

care to optimize. Figure 2.1 shows an overview of some well known OCI runtimes stack, implementing different isolation mechanisms.

Docker Containers. *Runc* [29] is the traditional Docker runtime and uses the host's OS mechanisms (namespaces, cgroups) to create network and cpu resources isolated from other processes. It then mounts the bundle in a specific path on the host and calls chroot in order to run the binary. Cgroup containers are lightweight like processes - they are processes - and present fast startup times. By leveraging the host's page cache runc containers can efficiently increase performance. However, process isolation might be lightweight but it comes at the cost of security [67]; multiple containers share the same kernel composing a possible risk, especially in multi-tenant environments.

VM sandbox techniques (e.g QEMU, Firecracker, Unikernels) present other low level runtime implementations. The runtime configures a sandbox - factory consisting of the hypervisor, network devices, block devices and other configuration needed in order to bring up the guest. The container rootfs needs to be plugged in the Hypervisor, and then, usually, an agent inside the guest setups the rest of the configuration and starts the task.

Kata Containers runtime. *Kata-runtime* sets up a Firecracker micro VM or QEMU Full VM sandbox and runs the container inside it. Kata-runtime is spawn as separate process by the higher level containerd runtime and is implemented in compatibility with containerd. It communicates via grpc protocol [14] with a long running agent spawned as the guest init process, and is responsible for plugging the container rootfs to the microVM virtio block device. The agent receives the container creation request, mounts the container rootfs inside the guest and starts the entry process. The rest of the lifecycle is run controlled by sending grpc requests to the agent.

Nabla Containers runtime. *Runnc* executes a Rumprun unikernel [55] packaged in a Nabla Container image. Nabla combines the unikernel concept with benefits of the container ecosystem. Apart from running a unikernel as a lightweight guest VM using KVM [94], Nabla containers alternatively introduce a seccomp confined sandbox [95] to dynamically load the unikernel into their own address space, effectively running it as a system process. This approach eliminates hypervisor-based virtualization overheads, facilitates debugging by allowing the use of common tools and enables sharing of the unikernel memory itself. However, the monitor is now responsible for ensuring the unikernel is confined.

Similarly to traditional VMs the unikernel, e.g. Rumprun, provides all the needed bits to build and run a generic POSIX-y application such as a serverless function. The Virtual Machine Monitor handles the loading of the unikernel and the actual block/network I/O. A Nabla container image is built like a traditional container. It consists of the unikernel binary as the base layer. A number of other layers are added on top of it, including configuration files and other dependencies. *Runnc* is responsible for creating the container from the image and run it. A filesystem view of the container compatible with the unikernel needs to created.

Other well-known runtimes. Firecracker-containerd [9] follows a very similar approach with Kata. However, the communication [5] with higher level orchestration frameworks such as Kubernetes is not yet implemented. Runsc [28] is a runtime for containers inside the gVisor sandbox. gVisor [12] acts like an application kernel for the container, providing OS functionality with reduced host kernel access.

2.2.4 Container storage mechanisms

The high level OCI runtimes implement the storage drivers responsible for applying the layer archives, respectively to the image manifest, in a runtime bundle compatible with the low level runtimes (2.2.3.2). Docker graphdrivershave been historically serving this purpose as part of the Docker daemon. In the last few years, with the rapid growth of Kubernetes and its Container Runtime Interface (CRI) [5], the need for more flexible and plugable approaches quickly evolved. Containerd runtime started to replace Docker as a more modular design [8]. Snapshotter is the Docker graphdriver equivalent for containerd. In contrast with the image oriented approach of Graphdrivers, the Snapshotter API is layer oriented by introducing the concept of Snapshots [6]. Thus for the rest of the thesis, any references to the layers are implying the Snapshot equivalent definition. The backing implementation of a Snapshotter utilizes the Linux kernel mechanisms and filesystems in order to create a new layer from an extracted archive.

2.2.4.1 File based storage

File Based approaches such as Overlayfs [22] are based on union mount filesystemsin order to project a unified view of a layer along with its ancestors. The container rootfs is a single union mount point of the container writable layer on top of the read only image layers. As a result the layer files are not only shared among containers based on the same image but they are also efficiently cached in host's page cache, significantly reducing the I/O operations and increasing the I/O speed. However the Copy-on-Write (CoW) mechanism that is used to track changes on image files - residing in lower read only layers - operates in the file granularity; copying the entire file in the upper layer, introduces I/O writing overhead and block duplication.

2.2.4.2 Block based storage

Block based storage drivers for containers, like the Devicemapper, prepare the container rootfs at the block level. The CoW operations copy only the dirty blocks on the upper device instead of the whole file, avoiding the file-based CoW overhead and I/O redundancy. This is achieved by using the Linux Kernel device-mapper driver, with the thin and snapshot volume targets [17]. The Devicemapper [1] snapshotter creates an empty base layer as a thin volume and each new layer is created as a device-mapper snapshot of its parent. In this case the container filesystem is a single CoW device snapshot of the upper image layer. However although this mechanism introduces well performed CoW, each container maps to a different device-mapper device and as a result the cached data cannot be shared among different instances causing a significant degradation in cache performance.

2.2.5 VM storage mechanisms

2.2.5.1 VM Block based storage

The traditional storage driver for Virtual Machines is the virtio-blk device. The state-of-the-art hypervisor used in Serverless systems, i.e. Firecracker, is relying on the virtio-blk driver for guest storage. The virtio-blk frontend driver (in the guest kernel) places read, write, and other requests onto the virtqueue, so that the backend driver (in host) can process them accordingly. Communication between the frontend and the backend is based on the virtio kick and notify mechanism.

Similarly, unikernels, e.g. Rumprun, rely on the virtio-blk like driver for accessing data on the physical disk. Rumprun's root filesystem is backed by user-provided images with common

[1] Note that the kernel device-mapper snapshot targets are real CoW device snapshots. The containerd Snapshots are a layer abstraction and Devicemapper is a snapshotter implementation around the kernel real device-mapper.

filesystems (i.e. ext2, iso etc.). At boot time, the unikernel creates mountable block device mappings to host images. Using the ETFS (ExtraTerrestrial File System) [27] interface, Rumprun registers the images to generate corresponding block devices. With those devices in place, it can then mount the images inside the unikernel using common filesystems (i.e. ext2). In the Nabla case, the actual read/write calls from/to the images are performed by the Solo5 monitor. After the unikernel filesystem has been set up, the Solo5 tender mediates access to the actual host images. For any read/write to an image, Rumprun passes control to Solo5 through the proper bindings (rumpuser hypercall interface). Solo5 ensures confined access to the host image. In this end-to-end mechanism, the unikernel (Rumprun) interacts with user data files contained in the image, and the tender (Solo5) in the host manages the block level access to it.

2.2.5.2 VM File based storage

Virtio-fs/9p [24,25] introduced sharing filesystem directory trees with VMs. This is accomplished through a set of virtio devices that facilitate efficient communication between the host and guest. When a shared directory is configured, the virtio subsystem creates a virtual filesystem (VFS) on the guest that interfaces with the host's actual filesystem through a communication channel. Virtio-fs is using the virtio vhost interface for creating a direct communication between the guest and the host kernel.

2.2.6 Container storage drivers and VM compatibility

The file based drivers are natively incompatible with VM sandboxing as they rely on the host filesystem semantics. The gap is inevitable: the runtime bundle resides on the host filesystem but the workload will be executed inside the guest isolated address space. In order to brigde this gap there are filesystem implementations [24,25](Section 2.2.5.2) that expose the host filesystem inside the guest. Although these approaches are convenient there are strong security concerns as they rely on the vhost mechanism which directly creates a channel between the untrusted guest and the host kernel. Firecracker - a state of the art VMM used in Serverless scenarios- not only excludes virtiofs/9p from its design but it also eliminates any vhost functionality from its code base [23]. It is indeed a valid question whether the acceptable performance sacrifice (sandboxing containers inside VMs) in the name of security should be undermined by a possible expansion of the attack surface introduced by such mechanisms.

As a result, low level runtimes that use the ligthweight Firecracker VMs, e.g kata and firecracker-containerd, need to prepare the container bundle as a block device that can be plugged to a VM block based storage backend (§ 2.2.5.1). Both of these runtimes are relying on a block based driver (§ 2.2.4.2): the containerd Devicemapper snapshotter.

Unikernels compatibility. In the case of unikernels runtimes, such as Nabla, storage handling requires a combination of actions from both unikernel and the sandbox monitor. Conceptually the Devicemapper driver is combatible with Nabla runtime. However the current implementation of runnc creates a filesystem in an iso file (during container creation) containing all the container's files. This iso image is then provided as the root filesystem to the Rumprun unikernel. This approach adds an important overhead at boot time and incommodes the deduplication of common layers among multiple containers.

2.3 Serverless isolation models

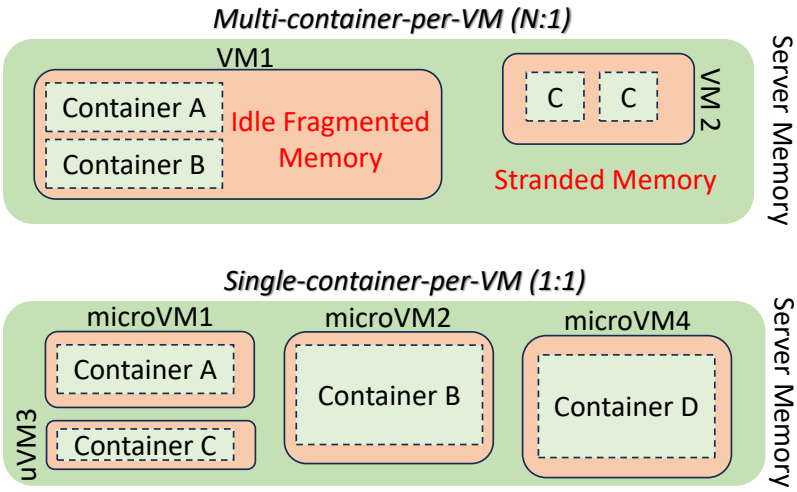


Figure 2.2: The 1:1 and N:1 serverless models.

FaaS providers use two main models [65] to securely deploy function instances into virtual machines: i) the multi-container-per-VM (N:1) [3,74] and ii) the single-container-per-VM (1:1) [2] model. Figure 2.2 provides an overview. While both models are adopted by the industry [2,3], each presents its own trade-offs with respect to i) cold start delays, ii) resource sharing, and iii) resource allocation. We discuss this in detail in § 4.1.

2.3.1 Single function per VM

The single-container-per-VM model deploys each function instance in a lightweight VM, i.e., a microVM, and scales up the number of VMs during bursts of requests.

2.3.2 Multiple functions per VM

The multi-container-per-VM model scales up the number of function instances (containers) of the same user within the same VM to handle spikes in incoming load.

2.3.3 Trade-offs

2.3.3.1 Cold starts

Spawning a function instance to serve incoming requests comes with significant cold start delays [53,54,86,92], as a new sandbox (container) must be created and the new instance's runtime dependencies and private state must be initialized. This is the reason why providers typically cache idle instances for fixed time windows (keep-alive) [47,86] or delay spawning new instances as much as possible [57], in an attempt to minimize cold start invocations. The 1:1 model adds the extra non-negligible overhead of booting a new microVM to spawn a new instance. While a long line of research attempts to optimize away boot costs [36, 60, 88, 90, 92], starting a VM is in principle a more expensive operation than creating a new container [48, 66]. Moreover, the N:1 model can potentially avoid some of the runtime initialization overheads, as the dependencies of the function are typically already warm in the shared VM, i.e., already cached by the guest OS.

The multi-container-per-VM model (N:1) avoids the microVM boot overhead per new instance.

2.3.3.2 Resource sharing

Another trade-off is related to memory footprints. The 1:1 model introduces higher memory overheads as it requires a separate guest OS kernel and function runtime per function instance. On the other hand, the N:1 model shares the OS and runtime dependencies among the instances that are co-located within the same VM. For example, the function's language libraries are cached by the guest OS and reused across instances.

The multi-container-per-VM model (N:1) has lower memory tax per function.

2.3.3.3 Resource allocation

One of the key aspects of serverless computing is that the user demands a specific maximum amount of CPU and memory resources for each function to execute [53]. This allows the provider to optimize the resource allocation on the server side, based on these fine grained, predictable, and predefined requirements.

In the single-container-single-VM model (1:1) the resources allocated per microVM closely follow the target function's resource limits. The allocation is simplified, the function size along with the sandbox overhead (e.g., microVM and container memory requirements) are calculated as a single unit that needs to optimally fit in a set of available server nodes. More importantly when the provider decides to scale down function resources, it can shut down a microVM and instantly release a predictable amount of memory.

On the other hand, on the multi-container-single VM model (N:1), the resource allocation is more complicated and subject to the concurrency factor, i.e., the number of function instances provisioned to be deployed within the same VM. For this model, VMs are typically created with enough memory to accommodate multiple instances. However, as the number of required instances varies over time, e.g., based on the incoming load, this model frequently results in memory fragmentation [65, 66]. Figure 2.2 shows how allocated VM memory is idling when the load is low and the number of instances has been scaled down by the serverless runtime. The VM still ties down its maximum resources in the host. As the VMs are now larger, they exacerbate memory stranding [46, 64]. Figure 2.2 shows how overprovisioned VMs can lead to wasted server memory.

The multi-container-per-VM model ties down idle resources when the load is low.

2.4 VM Memory Elasticity

In this section we discuss the overheads involved in resizing a VM's memory using vDIMM-based memory hotplugging [50, 68]. We focus on vDIMM hotplugging as it is the mechanism employed by virtio-mem [50], the state-of-the-art for dynamically adjusting VM memory. We discuss other interfaces, such as ballooning [20, 37], in Section 6.1.

2.4.1 Memory hotplugging

Memory hotplugging dynamically adjusts VM memory by emulating in the hypervisor the hotplugging of physical memory DIMMs. The VM is notified, e.g., via ACPI for x86 VMs, about the hotplug operation and then relies on the OS to add or remove the DIMM memory to the system. We elaborate on the mechanisms used by Linux to add (online) or remove (offline) DIMMs in the next paragraphs. The default DIMM hotplug interface is tailored for (un)plugging physical memory, and thus has various inherent limitations. It only operates in large (DIMM-sized) granularities and requires architecture-specific support. virtio-mem [50] addresses these limitations by introducing a paravirtualized hotplug interface. It exposes a new device, i.e., a paravirtualized DIMM, which it chunks up in smaller slots. Each of these slots can be plugged and unplugged independently, thus allowing the adjustment of VM memory in smaller quanta. virtio-mem relies on the existing native Linux mechanisms to online or offline each slot. Although virtio-mem improves upon the default DIMM hotplug, multiple works [46, 51, 69] show that DIMM memory hotplugging, with or without virtio-mem, suffers from overheads stemming from page migrations.

Linux manages physical memory in the granularity of pages (i.e., 4KiB) but adds and removes ((un)plugs) memory in the granularity of memory blocks, i.e., 128MiB for x86.

Hotplugging a memory block involves two major steps: a) add the memory block to the OS (*hot-add*) and b) expose it to the OS allocator (*online*). *Hot-add* updates the OS metadata for physical memory, such as the Linux memory map array (*memmap*), and is penalized by the overheads of creating and initializing the *struct pages* that represent each 4KiB range of the block. *Online* releases these pages to the OS allocator, to expose the plugged memory to the system as usable.

Hotunplugging a memory block involves two major steps as well: a) retract the pages of the block from the OS allocator (*offline*) and b) remove the block from the OS by destroying all the corresponding metadata (*hot-remove*). The first step dominates unplug performance as it has to isolate the pages of the block to be unplugged and this commonly involves migrating occupied pages.

> *Plugging memory is dominated by initialization overheads and unplugging by page migrations.*

2.4.2 Memory initialization

Zeroing-out memory blocks can affect plug or unplug performance based on system-wide configuration. The OS needs to zero out memory for security and correctness reasons. For example, multiple user-space libraries (e.g., libc) assume that newly allocated memory is zeroed-out. Linux has two modes to guarantee this: a) it either zeros out memory pages during allocation (*init_on_alloc*), or b) release (*init_on_free*). This is a boot-time system-wide configuration with implications on (un)plug performance. *Init_on_alloc* penalizes unplug operations, as unplugging uses generic allocation routines to reserve the memory to be *offlined* from the rest of the system. *Init_on_free* penalizes plug operations as pages need to be zeroed out before they can be released to the OS allocator for *onlining*.

> *Zeroing out memory blocks penalizes (un)plug operations.*

2.4.3 Physical Memory Allocation

Physical memory allocation indirectly affects unplug performance, as it distributes occupied pages over memory blocks. Firstly, the OS memory allocator can place non-movable data [82, 102], e.g., kernel memory, on multiple memory blocks, which would prevent them from offlining. To address these issues, Linux has introduced a special memory zone (ZONE_MOVABLE), to distinguish movable and non-movable pages. ZONE_MOVABLE serves all allocation requests that

correspond to data that in principle can be migrated, e.g., user-space allocations and file system caching layers (i.e., page cache). The memory blocks that populate this zone can always be successfully offlined, as long as there is enough free memory in the rest of the system to migrate their occupied pages. Linux uses other zones, e.g., ZONE_DMA and ZONE_NORMAL, to serve allocations for unmovable data and to fall back for movable data when ZONE_MOVABLE is full. *Memory that is hot-plugged is typically added to ZONE_MOVABLE to guarantee the success of any future unplug operations.*

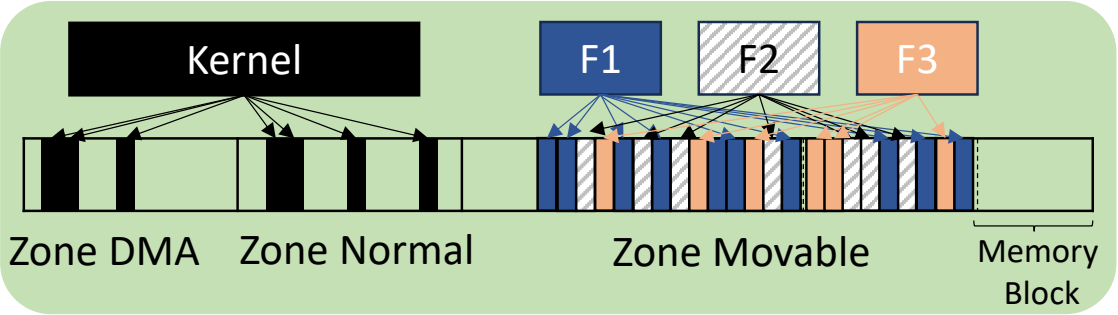


Figure 2.3: Processes footprints interleaved within a single physical memory block.

While ZONE_MOVABLE guarantees that offlining will succeed, it does not necessarily speed it up. The OS commonly interleaves the footprint of different processes [32,97] on multiple blocks of physical memory during memory allocation. This leads to expensive migrations of interleaved pages when the OS tries to reclaim the memory of one of the processes that exited.

Let us consider a common scenario when it comes to virtual machine resizing. Figure 2.3 depicts a VM with 3 running processes. The OS has mapped their virtual address spaces only with pages from ZONE_MOVABLE, but their footprints are interleaved across multiple physical memory blocks inside the zone. Linux allocates memory lazily, at the page granularity (4KiB or 2MiB), when the processes access their pages for the first time (page fault). To serve the faults, the OS allocates any available page, hindering contiguous mappings and scattering the workload's footprint across blocks. This is the reason why the depicted applications footprints are interleaved.

This interleaving eventually penalizes memory unplugging. For example, in Figure 2.3, when the process *F2* terminates and releases its memory pages, the serverless runtime would attempt to down-size the VM to reclaim the released memory. The hypervisor sends a request to the guest OS to unplug resources equal to the size of the terminated process footprint. The guest will try to isolate (offline) memory (e.g., via virtio-mem) that matches the requested size by iterating over the guest's physical memory blocks. Due to the interleaving of process footprints, most memory blocks in the system will be at least partially occupied, thus the guest has to a) migrate

some pages to isolate enough memory blocks and b) then offline and unplug them. This can significantly penalize unplug performance.

> *Memory allocation interleaves processes footprints within the same memory blocks, indirectly penalizing unplug performance.*

2.4.4 CPU interference

Hot-unplugging memory can penalize the performance of processes running in the VM that is being down-sized. Related work [50] has shown that the migration of occupied pages hogs precious CPU resources interfering with other processes.

> *Page migrations can penalize the performance of other processes that run inside a VM which is being down-sized.*

CHAPTER 3

Storage for functions in VMs

This chapter presents Deverlay, a container storage driver solution designed to efficiently carry the traditional containers benefits to the virtualized containers new era. Deverlay natively supports the virtualization interfaces for storage as it creates containers based on block volumes. While Deverlay currently targets Functions sandboxed in containers in VMs, the design idea is able to support native containers or other sandbox mechanisms, e.g., unikernels (ch. 2) as described in a experimental prototype for unikernels in the last section of this chapter. Table 3.1 provides a brief description of the constituent components of Deverlay.

The first section of this chapter motivates Deverlay via a comprehensive evaluation of the container startup latency when using different container storage solutions for traditional host

Component	Purpose	Main Contribution
Prepare Layer	Recursively construct the function's filesystem layers	Block level container image layers Support layer share-ability among containers
Provisioner	Maintain a pool of pre-created empty block devices	Shift device creation overhead outside the execution path
DeverlayMount	Container filesystem creation	Enables shareable container layers among multiple VMs

Table 3.1: Deverlay Components

containers(§ 3.1). The results showcase the performance advantage of file based (§ 2.2.4.1) over block based drivers (§ 2.2.4.2), motivating the development of Deverlay for containers running inside VMs.

Section 3.2 introduces the Deverlay principles and goals and continues with the mechanism design. Starting with the design overview (Section 3.2.1), the next sections provide an in-depth description of the design and implementation of Deverlay. Deverlay first enables the preparation of block based containers from block based container image layers for both native and virtualized execution (Section 3.2.2). The *Provisioner* (Section 3.2.3) and *DeverlayMount* (Section 3.2.4) are described next. *Provisioner*, shifts the empty block devices creation overhead outside the execution critical path. Finally *DeverlayMount* creates the container filesystem by union mounting the container layers in the right order after - in case of VM sandboxing - hot-plugging the block device layers.

Deverlay is designed and implemented as a containerd Snapshotter driver, targeting functions sandboxed in containers in Virtual Machines (§ 2.2.2.2). The experimental evaluation of Deverlay (Section 3.3) on an Intel Xeon server shows that startup latency of a single instance (§ 3.3.2.1) is reduced by 13.21%-40% for virtualized and native containers respectively. Deverlay supports cache sharing between multiple containers and efficiently reduces cache buffers usage by 96.09% compared to Devicemapper, improving memory usage when running multiple instances on the same node (§ 3.3.3). By eliminating disk I/O during container startup (§ 3.3.4), Deverlay can boot 80 virtualized container instances from the same image in less than 4 seconds following similar exponential scaling behaviour both for native and virtualized execution (§ 3.3.2.3).

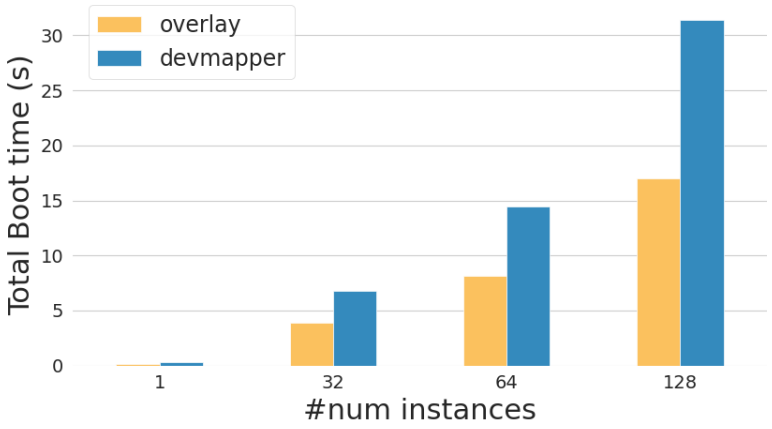


Figure 3.1: Startup latency for multiple host containers from the same image

3.1 Motivation

As explained in § 2.2.6 the state of the art sandboxing runtimes for Serverless are using block based storage drivers (§ 2.2.4.2), e.g. Devicemapper, for compatibility and security reasons. Compared to file based drivers (§ 2.2.4.1), e.g. Overlayfs, the Devicemapper needs to create a new device for each new container and additionally lacks the ability to share common cached data among different containers.

Horizontal scaling, by spawning multiple worker - instances of the same image in parallel in order to efficiently serve incoming requests, is very common in the concepts of Serverless or Cloud native. In such cases, fast instantiating is critical and data duplication or redundant I/O load can introduce a bottleneck. We compare the two drivers in this scenario.

As shown in Figure 3.1, the Devicemapper's inefficiency causes significant boot overhead when running multiple containers from the same image compared to file based Overlayfs. Devicemapper needs almost 15s to start 64 instances of nginx while Overlayfs achieves it in less than 10s.

As shown later in Section 3.3, creating a new block device (snapshot) at each instance boot introduces a high number of I/O operations and, as a result, there is an important effect on startup times. Moreover, the fact that cached data on memory are not shareable among instances not only leads to multiple caching and further increase of I/O redundancy, but also eliminates the potential benefits of faster subsequent startups leveraging cache hits.

Overall, the existing state-of-practice container storage drivers have been inherited from the traditional Docker age, when the container workloads were deployed directly on the same host as the Docker engine. In the new era, of the Cloud Native and Serverless, providers use virtualized secure containers in order to deploy serverless services and functions. Related research work

(Section 6.4) mainly focuses on optimizing aspects of the CoW operation on the file / block level drivers for host containers or faster image distribution across clusters, without evaluating on the new trend of secure containers like Kata (§ 2.2.3.2). Memory efficiency and I/O redundancy by leveraging the host caches is not yet carefully implemented and evaluated.

3.2 Container storage for VMs

We design and implement *Deverlay*, a container storage driver targeting containers running inside Virtual guests for Cloud native multi-tenancy architectures. *Deverlay* goal is to keep up with the hardware virtualization security standards by keeping the host filesystem semantics from being exposed inside the guest; thus, design solutions like virtiofs/9p (2.2.5.2) are out of the question. We design a block based driver that can overcome the existing state-of-practice - Devicemapper - inefficiency. Furthermore, targeting generic widely used mechanisms such us virtio block devices covers a larger surface of potential use cases with different hardware virtualization technologies, not only limited to Virtual Machines but also Unikernels: a technology that will further improve FaaS and Cloud native overall experience and security. *Deverlay* meets the following goals:

Fast boot times *Deverlay* eliminates the block device based container preparation overhead and as a result reduces the sandboxed container boot time. The remaining boot overhead depends on the sandbox initialization technology. As the isolation mechanism becomes more lightweight and the sandbox initialization faster, the *Deverlay* added potential value is increased. Passing from full blown Virtual Machines like QEMU, to microVMs like Firecracker and to even faster unikernels like Nabla (§ 2.2.3.2), the *Deverlay*'s stable and low preparation overhead converges into lightweight "cgroup - like" spawning with file system optimized drivers such as Overlayfs. When compared with the Devicemapper, *Deverlay* achieves greater reduction in the overhead between cold and warm boots, by efficiently sharing the cached blocks of different container instances. Moreover, it further optimizes the container creation latency by implementing a pool of available devices managed by a *Provisioner* component for fast device consumption.

Scalability The Devicemapper driver is totally inefficient when it comes to horizontal scaling as it introduces high level of redundant I/O at startup time, which inserts the limit in parallel spawning at a very low standard. *Deverlay* can efficiently exploit the potentiality in lightweight virtualization with Firecracker microVMs by booting 80 Kata Container instances in less than 4s, increasing the number of instances exponentially through time. It achieves similar behaviour in performance compared with file based drivers for host execution, despite executing over the hardware virtualization gap.

Cache Sharing & optimized I/O redundancy: *Deverlay* introduces to the block-level container storage drivers the cache sharing efficiency of file based drivers. It achieves that by mapping image layers to block devices and sharing the devices among containers build from the same layers. As a result our design can efficiently avoid reading the 97,51% of the data that the Devicemapper needed to read from disk when launching 128 Kata containers instances from the same image.

3.2.1 Overview

Deverlay is designed to optimize the root filesystem preparation for virtualized secure containers, focusing on host caches sharing and I/O redundancy for identical images or different images containing one or more identical parent layers. *Deverlay* is also compatible with traditional runc containers running on the host. The current prototype is implemented in the containerd engine respecting the Snapshotter API architecture [6]. *Deverlay* prepares (§ 3.2.2) each new layer on the host as a separate thin (Linux device-mapper) volume target. The block device preparation is further optimized by the *Deverlay Provisioner* component (§ 3.2.3). Each layer device contains the layer files extracted from the original layer tar file. *Deverlay* introduces a *DeverlayMount* struct (§ 3.2.4) describing a union view of each layer with its parents, that can be used both for cgroup containers (runc) or VM sandbox containers (Kata Containers, Nabla Containers, firecracker-containerd). Although *Deverlay*, at its core, is a block - based solution, its concept is hybrid: from the container engine point of view the container is a stack of layer block devices; from the container itself point of view the root filesystem is an overlayfs mount.

3.2.2 Layer and Container Preparation

Deverlay prepares and stores the images on the host. Each image layer is stored in an initially empty thin volume device. Each device contains only its corresponding layer files. Layer preparation starts with requesting an empty device from the *Provisioner* and mounting it on the host in order to extract the layer files from a downloaded tar archive. The `Prepare()` layer operation recursively constracts and returns a mount struct (§ 3.2.4) implying an overlay mount which on top has the empty device and on the base level an ordered stack of the parent devices. A new container is just an empty new RW layer. In order to create a new container, from an existing image, the `Prepare(parent)` method is invoked where parent is the upper layer of the existing image, as shown in Figure 3.2.

3.2.3 Volume provisioning

Preparing a new volume device is not cheap in terms of time. As we show in Section 3.3 it is costly in term of I/O traffic, causing an important overhead at startup time, especially in parallel spawning cases. *Deverlay* shifts this overhead outside the critical path of execution. Instead of creating a new device at the container preparation step, *Deverlay* consumes an asynchronously pre-created device from the *Provisioner* pool. As shown in Figure 3.3 the *Provisioner* implements two pools:

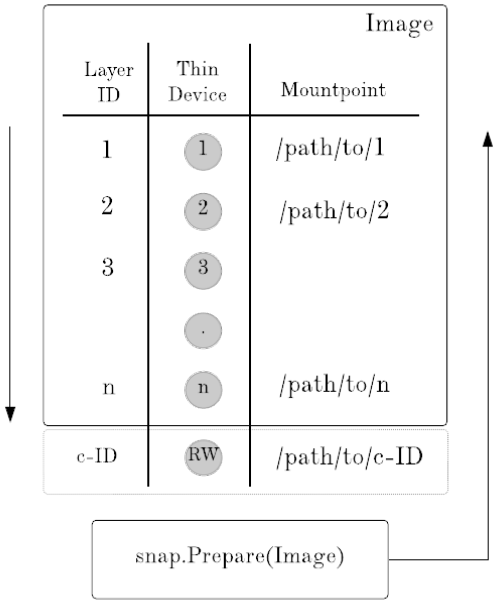


Figure 3.2: Container preparation with the *DeverlayMount* struct

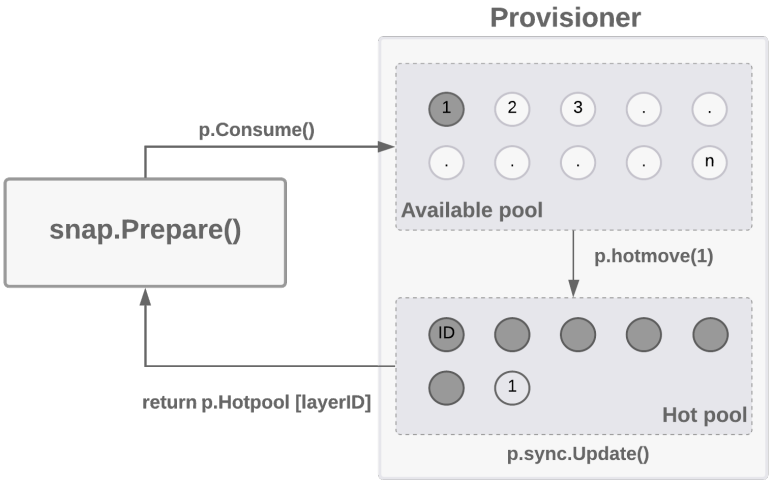


Figure 3.3: Overview of *Provisioner* design and workflow

- The **Hot Pool** is a Hash Map of devices currently in use. The key is the corresponding layer ID and the value is a pointer to a device struct with useful info as the mountpoint on host, the device path etc.

- The **Available Pool** is a list of devices ready to be quickly consumed.

When *Deverlay* prepares a new layer it calls the *Provisioner* `Consume()` method. A pre-created device is moved from the Available pool to the Hot Pool. The array and the Hash Map are safely updated using mutexes.

The *Provisioner* initializes the Available pool at system's startup, reading the desired size - number of devices - from the configuration file. During the Initialization step the *Provisioner* creates the Available pool of block devices. Each block device is created as a thinly provisioned empty logical volume using the Linux Kernel device-mapper functionality. The backing thin-pool can be deployed either by using the Logical Volume Manager (LVM) [?] with physical disks/SSDs for better performance or by using the loop device mode backed by raw images files [?]. The *Provisioner* can easily implement policies for re-initializing the Available pool in order to keep the number of available devices always above a desired threshold, by asynchronously creating new devices or / and re-formatting removed layer devices from destroyed containers and deleted images.

3.2.4 Filesystem Mount

The *DeverlayMount* is an object that describes how to mount the container image layer devices in order to prepare a container rootfs view. As shown in Figure 3.2, it consists of a list that maps each container layer device with its respective mountpoint. The list is ordered from lower to upper layer level where the last mapping is the container RW device layer. The object is passed to the lower level runtime (e.g. runc, kata-runtime) which is responsible for performing the actual mount before executing the container entrypoint.

In the first case, when running runc containers the parent layers - devices are already mounted on the host. Only the container RW layer device returned by the *Provisioner* `Consume()` method needs to be mounted before performing the overlay mount on top of the image. That means that *Deverlay* with runc containers is invoking one extra mount operation compared to Overlay or Devicemapper storage drivers. This extra operation overhead compared to Devicemapper device creation (snapshot) is totally dominated by the benefits of the *Deverlay Provisioner* Available Pool. Mounting an already created device containing an empty filesystem is much cheaper that creating a block level CoW snapshot of a non empty device, like the Devicemapper does. The `DeverlayMount.Mount()` method implements the final union mount of the container rootfs path using overlayfs over the devices mountpoints, like the Overlay driver would do.

In the second case, when running virtualized secure containers, the *DeverlayMount* object is passed to the low level runtime (e.g. kata-runtime). The runtime plugs the block devices listed on the second column of Figure 3.2 to the Virtual Machine, using the Hypervisor block driver.

It then replaces the Mountpoint columnn with the appropriate guest paths; the device column is updated with the virtual block device path in the guest (e.g /dev/vdX). The updated object is encapsulated in the container create request and sent to the agent running inside the VM. The agent performs the final mounts inside the guest, invoking the `DeverlayMount.Mount()` method, and starts the container.

The *Deverlay* Snapshotter design is natively compatible with any Virtual Machine Monitor supporting generic block storage drivers. *Deverlay*'s admission and key design concept is that for secure containers the filesystem representation should be based on the OS mechanisms of the Guest, as the virtual environment is at the end the real execution environment. In the following section we evaluate the implementation of *Deverlay* on containerd intergrated with the Kata Containers [15]: an Open Source community leading the way on secure container workloads deployment, supported by the Open Infrastructure Foundation [21].

Native Environment	
Processors	2-socket Intel Xeon CPU E5-2630 v4 (Broadwell) 10 cores/socket, 2 threads/core, 2.2GHz
NVMe	Intel® SSD DC P3600 Series 400G
Memory	256G (128G per socket)
OS	Debian Linux v5.10
Kata Container Environment	
VMM	Firecracker (KVM) v0.23.1 1vCPU
Memory	2G
Runtime version	2.3.0

Table 3.2: Deverlay Evaluation System Configuration.

Workload	Description	Use Case
nginx	web server [?]	Serverless Service
python hello	function timestamp print [?]	Serverless Function

Table 3.3: Deverlay Evaluation Workloads

3.3 Evaluation

3.3.1 Methodology

Experimental Setup. We implement Deverlay for containerd v1.6.0 and evaluate it on Debian Linux Kernel v5.10 host. For hypervisor - level sandboxing, we use Kata Containers v2.3.0 with Firecracker (KVM) v0.23.1 configured with 2GiB of memory and 1 vCPU for each instance. For the evaluation, we use a 2-socket Intel Xeon CPU E5-2630 v4 (Broadwell) with 10 cores/socket (2 threads / core), each with 128GiB of memory. To minimize jitter, we pin each Firecracker guest on cores of the same socket and set the core frequency to 2.2GHz. In cases of over-provisioning (more instances than number of cores) we equally load all the host CPU threads. We also run some of the experiments with os - level sandboxing using cgroup host containers - runc runtime - in order to further breakdown the results and show that Deverlay can keep up with the performance of the file based drivers when running on host, while outperforming Devicemapper in all scenarios. The Deverlay and the Devicemapper storage driver volume pools are setup with LVM on NVMe in order to avoid the performace issues of traditional HDDs. The Overlayfs storage driver is setup on the NVMe using ext4 filesystem layers. The experiment system environment is shown in table 3.2.

Workloads. We choose two different types of Serverless applications: a web server deployed as a container image (service) and a simple python code snippet (Faas) that boots and immediately prints a timestamp. Table 3.3 provides a description of the evaluation workloads.

Performance Metrics.

- Boot time: We measure the boot time for each workload in milliseconds. The nginx boot time represents the time elapsed from issuing the run command until the nginx worker thread has started. In the python example boot time is the difference between a timestamp when issuing the run command and the timestamp printed from the python function that serves as the container entry point.
- Cache usage: We report the cached data size using the vmstat tool in order to show that Deverlay leverages the cached data in host page cache from other container instances.
- I/O usage: We measure the physical disk accesses using the iostat tool in order to show that Deverlay avoids the redundant I/O accesses of Devicemapper.
- I/O performance: We use dd to measure the read performance from inside the container.

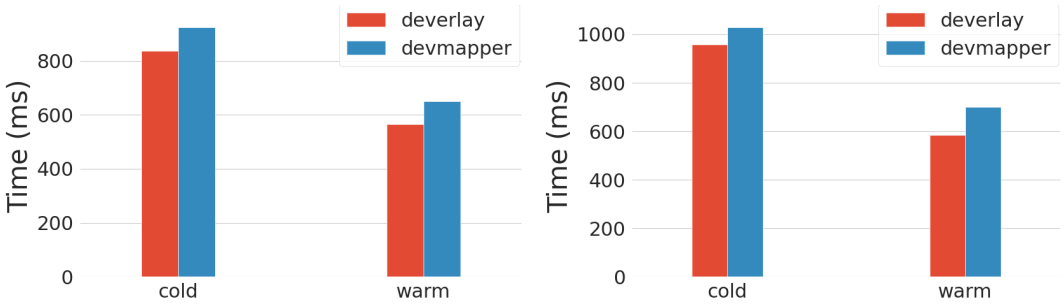
Evaluation scenarios. We use Devicemapper storage driver as the baseline driver for Kata Containers. We use the Overlayfs storage driver as the baseline in the experiments that include execution of containers on host. We refer to the VM sandbox related binaries and libraries, e.g. guest kernel and OS, as the sandbox runtime. We present results for the following scenarios:

- Single instance: we evaluate boot time in two cases: a) when an instance is booted for first time and the sandbox runtime is not present in the host page cache - we refer to this as cold start, and b) when an instance is booted with the sandbox runtime present in the page cache from a previous run - we refer to this as warm start. Warm excludes the overhead of bringing the sandbox runtime from disk and the Deverlay's improvement is more clear: normally the sandbox runtime is common for multiple different users and workloads on the server and it is expected to be present in server's page cache.
- Multiple instances: we evaluate boot time while the number of instances running on the system sequentially increases. We also report the page cache usage and the I/O accesses and show that Deverlay efficiently shares common layers between multiple instances.
- Concurrent boot: we spawn batches of containers in parallel in order to evaluate container boot time and batch total boot time when scaling is needed, e.g. there is a load spike for the application.

3.3.2 Boot Time

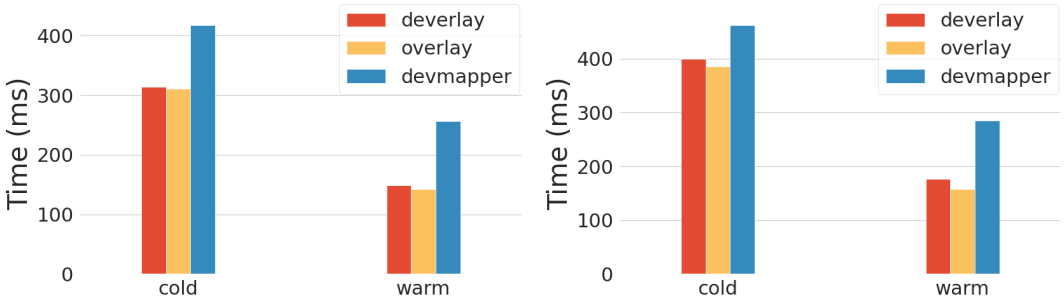
3.3.2.1 Single Instance scenario

In the first experiment (Figure 3.4) we measure the boot time for running one single instance of a Kata Container.



(a) Web server (nginx) (b) Python Function

Figure 3.4: Single instance startup time for VM container.

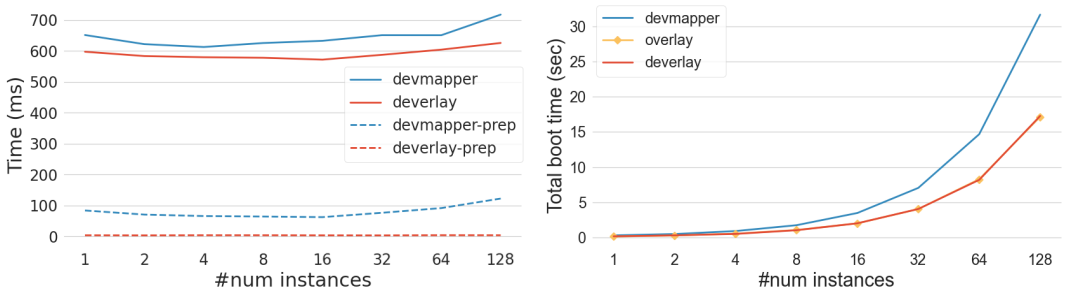


(a) Web server (nginx) (b) Python Function

Figure 3.5: Single instance startup time for host container.

In Figure 3.4a we present the results for the Kata Containers case. We observe that with Deverlay storage, starting the first nginx instance takes 836ms (cold: sandbox runtime not cached) and the second 565ms (warm: sandbox runtime cached from previous run). That's 9.62% and 13.21% faster than the Devicemapper respectively. Similar results for the python function are shown in Figure 3.4b.

As explained in 3.3, the first cold run also includes the overhead of bringing the Guest kernel & rootfs binaries to the host page cache. In order to better investigate the difference between the cold and the warm startup of Deverlay without taking into account the "noise" of the sandbox binaries caching time overhead, we execute the same experiment for host containers and compare our implementation with the Devicemapper and the Overlayfs snapshotter. Figure 3.5 summarizes our results. We observe that Deverlay can cold start a python function on host in 360ms, 22.24% faster than the Devicemapper, as shown in Figure 3.4b. The second instance (warm) startup is 40% faster compared to the Devicemapper, as it efficiently uses the cached data from the buffers cache and only mounts a device from the Provisioner; while the Devicemapper creates a new snapshot device invoking I/O traffic without being able to share data with the other identical instance. We note that Overlayfs is 6.73% - 7.60% faster than Deverlay for cold and



(a) Container in VM

(b) Container on host

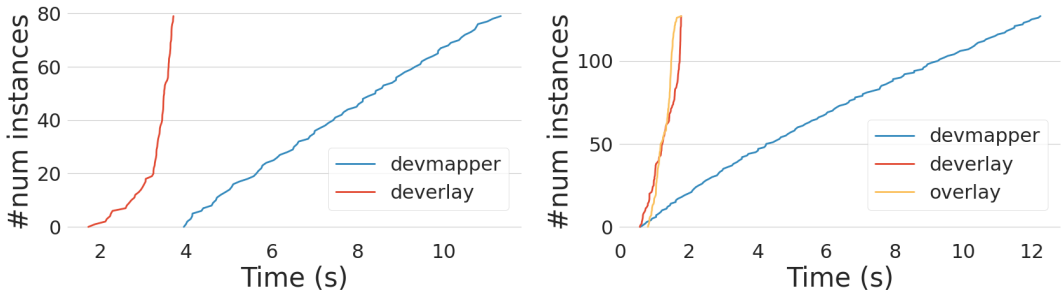
Figure 3.6: Sequential startup of 128 nginx instances.

warm startup as our prototype needs to mount the container device during the Prepare() step (as explained in § 3.2.4) before invoking the overlay mount operation. Overlayfs, on the other hand, has no Prepare overhead rather than creating a new container directory on the host before invoking the overlay mount. The nginx results from Figure 3.5a are similar. As we describe in section 3.4, there is room for further optimizations but our conclusion here is that Deverlay, even block-based, can efficiently reduce the startup time by reclaiming cached data from the page cache - like Overlayfs - and by consuming pre-provisioned devices from the Provisioner Available Pool.

3.3.2.2 Multiple Instances scenario

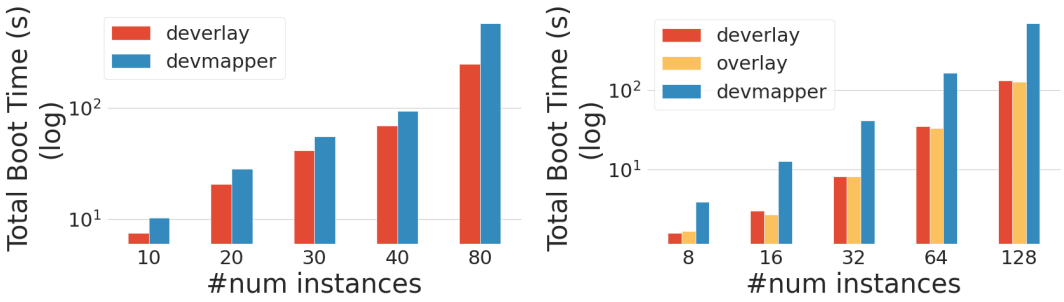
In order to investigate the boot overhead while the number of instances running on the system sequentially increases, we spawn sequentially 128 Kata containers and 128 cgroup containers from the same nginx image. Figure 3.6 summarizes the results.

We first breakdown the startup overhead for each individual Kata Container instance (Figure 3.6a) as the number of instances grows sequentially. We observer that the Devicemapper Prepare method adds a constant overhead on every boot, representing the cost for taking a device snapshot of the base layer. As the number of running instances on the system grows the overhead of this operation also increases as it invokes lots of I/O traffic - as explained in § 3.3.4. Deverlay manages to keep the Prepare method overhead stable and low as it benefits from the Provisioner Available Pool. Figure 3.6b shows the total startup time for sequentially starting host containers. Deverlay follows the performance of Overlayfs. As the containers running on the system increases, the total boot time increase rate grows slower for Deverlay than for Devicemapper and closely similar to Overlayfs. Deverlay has shifted the I/O tensive task of creating a new thin volume outside the execution path. The Devicemapper not only creates a new snapshot volume at boot but also invokes lots of reading requests, as this new device is not shared in cache with



(a) Container in VM (b) Container on host

Figure 3.7: scaling with parallel container spawning.



(a) Container in VM (b) Containers on host

Figure 3.8: scaling with parallel container spawning for different batch sizes.

the other instances. We show that I/O redundancy can become a bottleneck in scaling scenarios (§ 3.3.2.3 with parallel container batch spawning.

3.3.2.3 Scaling Concurrently scenario

In this experiment we spawn parallel batches of Kata containers and host containers from the same python image. All instances remain active until the end of the experiment and thus the moment that the Nth instance boots is reflecting a state where N-1 instances are already booted and running on the system.

Figure 3.7 shows the total number of instances booted as a timeline, for a parallel batch of 80 Kata containers and 128 host containers. Deverlay scales up exponentially-like in both cases, compared to Devicemapper linear scaling. By efficiently sharing cached data on the host buffers cache and fast device consuming from the Provisioner Available pool, our prototype is scaling 80 VM sandbox containers (Figure 3.7a) following the same exponentially scheme as containers running on the host with Overlayfs (Figure 3.7b). We note that the heavy I/O of Devicemapper is becoming a bottleneck from the start of the experiment - conccurrent I/O requests for 80

instances, and as a result even the first instance needs 4 seconds to start. Deverlay on the other hand can boot 80 instances even before the first Devicemapper instance. Furthermore Deverlay has a small overhead compared to Overlayfs, as shown in Figure 3.7b, when running on host and achieves similar scale up between virtualized and host execution - although the current prototype is designed and optimized for Virtual Machines. This overhead is caused due to the extra mounting operation, as already explained.

In Figure 3.8 we present the total boot time in log scale, for spawinng different parallel batches sizes of Kata and host containers. As shown in Figure 3.8a, Deverlay outperforms Devicemapper for all Kata batch sizes and manages to keep the growth rate of the total boot time significantly smaller. After the 40 instances Devicemapper starts to suffer from serious I/O bottleneck; the result for Deverlay is slightly affected only by the simple mount operation in the Prepare step. As the number of the Firecracker microVMs fully loads both CPU sockets, I/O operations become more expensive. In Figure 3.8b, for host containers, results show that Deverlay closely follows the Overlayfs performance keeping the total boot time in the same level while the Devicemapper starts to introduce overhead approaching the one order of magnitude difference even after the first 16 parallel instances.

We conclude that Deverlay is able to scale containers running in virtualized environments in the same way as containers running on the host. By reducing I/O redundant requests and sharing data from the host buffers cache, our prototype design manages to efficiently narrow the leap between the Guest boundary and the container image data residing on the host.

3.3.3 Memory footprint

We show that Deverlay can improve the Host Page Cache efficiency for containers running inside virtualized guests and as a result reduce the total memory footprint. In this experiment we use the *vmstat* tool to measure the cached data size when sequentially spawning 128 Kata containers from the same nginx image on the host. Figure 3.9 shows the results. Deverlay compared to Devicemapper efficiently avoids data duplication and reduces buffers cache usage by 96.09%. Buffer bar represents the caching data for host raw block devices - a ”page cache” for devices. Both Devicemapper and Deverlay are directly backed by the Linux device-mapper thin volume target and their cached data are reported by Linux as buffers. In the same figure the Cache bar represents the host’s filesystem page cache which at the start of each experiment run was also dropped. We point out that booting a secure container for the first time invokes an extra overhead not only due to misses related to the container image itself (buffer bar) but also for loading the Guest Kernel and rootfs binaries. We can notice that, by the fact that at the end of each experiment and after ”dropping” the caches in within, the page cache (bar) usage is almost the same.

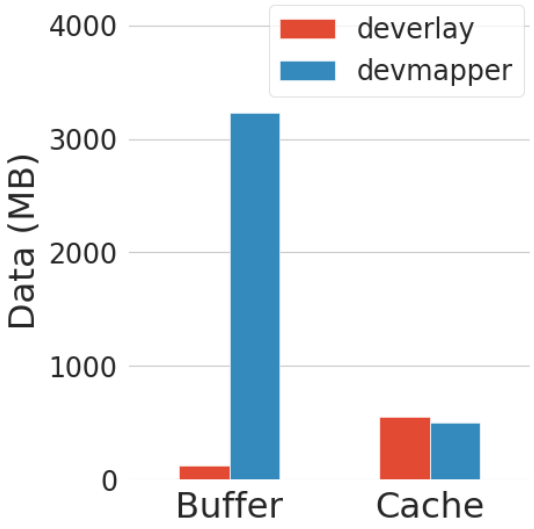


Figure 3.9: Total cache usage for different container storage drivers.

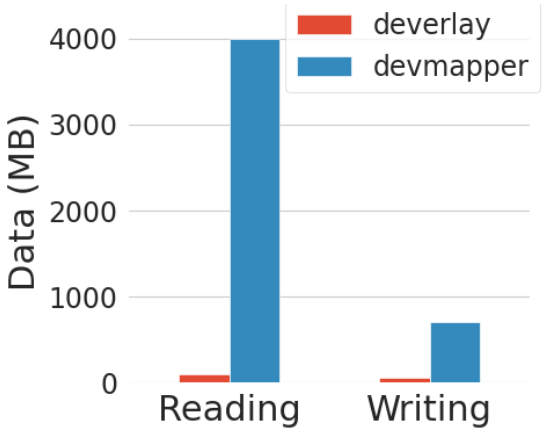


Figure 3.10: Data accesses on the physical disk for different container storage drivers.

3.3.4 I/O Redundancy

In this experiment we measure the physical disk accesses when starting up 128 Kata containers from the same nginx image. In figure 3.10 we show that Deverlay reduces the I/O redundancy by 97.51% for reading and 90.80% for writing data thanks to buffer cache sharing. The Devicemapper reads almost 4GB of data and writes 642MB to disk while the Deverlay Driver only reads 98MB of data and writes 59MB back to disk. We note here that although Deverlay is using an overlay mount inside the guest, meaning that the CoW operations are happening at the file granularity, the Devicemapper is finally writing a larger amount of data to the disk; we conclude that the total writing size is not only affected by the CoW operations caused by starting the container

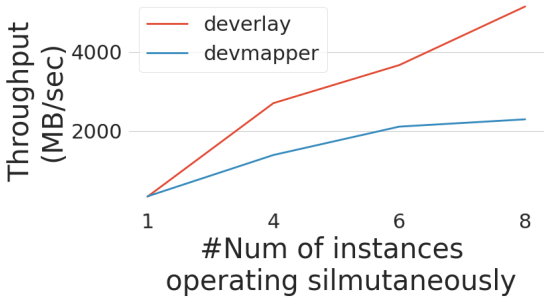


Figure 3.11: 64KB sequential read throughput by parallel VM containers from the same image.

task itself (e.g nginx). When creating a new snapshot of a block device a large amount of metadata are written to the disk. Deverlay eliminates this overhead by shifting it outside the critical path by consuming devices from the Provisioner.

3.3.5 I/O Performance

We show that sharing cached data among containers can efficiently improve the overall read throughput. We use *dd* to measure the read performance for sequential reads of 64KB from a file injected in the container image. In Figure 3.11 we progressively increase the number of Kata containers that operate simultaneously from 1 to 8, and we depict the throughput when maximum sharing is reached for Deverlay. Deverlay's performance scales almost linearly harnessing the benefits of host cache (DRAM) high bandwidth. Counterwise, Devicemapper cannot scale beyond 6 microvms, as the storage device bandwidth is throttled.

In the Cloud Native and Serverless contexts the workloads are running as stateless instances. Any persistence needed is covered by using remote storage volume drives (e.g. librados) RW mounted inside the container. It is even very common practice in Production environments to run containers with read-only filesystems. Similarly the mass data reads occurs usually through the network stack rather than the container filesystem itself. It is not inside the scope of this thesis to evaluate such solutions as they remain orthogonal to Deverlay's goal of fast and efficient scaling of VM sandboxed containers. However, there are cases, like Machine Learning applications running from images packed along with relatively small pre-trained models, where multiple instances are spawn across a cluster in order to perform parallel computations. In such cases we show that Deverlay can avoid redundant I/O load and maximize data read throughput by sharing the host cached data among different instances.

	Size. (MB)	Layers Number	Pull time Deverlay	Devmapper	Overlay
Ubuntu	27.2	1	1.458s	2.444s	1.514s
Python	171.9	6	11.627s	12.954s	10.317s
Nextcloud	322.7	20	22.548s	25.606s	19.709s

Table 3.4: Time to pull an image for different container storage drivers.

3.3.6 Image Pull

Image pulling is a very important operation especially in multi-node environments where multiple instances are spawn across nodes. In the Serverless use case, horizontal scaling by spawning new workers across the cluster can become inefficient if the container engine has high unpacking overhead. Similarly, in the microservices world fast pulling is not only important for scaling but also for quickly updating a cluster deployment with new versions of the services. Image pulling overhead is closely related to the Snapshotter. Each downloaded layer is unpacked - applied - on top of a new Snapshot derived by its parent. Thus, in order to unpack an image the container engine will invoke the Prepare method of the driver as many times as the number of image layers. As shown in table 3.4 [1] the Deverlay Driver is faster than Devicemapper in all cases, by leveraging the Provisioner devices. As the number of image layers increases the Devicemapper will become worse; as derived from Figure 3.6a the Devicemapper Prepare() method adds a constant overhead. In the traditional cgroup containers case the Overlayfs Snapshotter remains slightly faster than Deverlay, which is expected as its Prepare method is faster.

[1] Note that the download time is not included here, as the network bandwith is irrelevant in this comparison.

3.4 Discussion

3.4.1 Asynchronous Pool management

In the current prototype the Provisioner component is implemented inside the Deverlay Containerd Snapshotter plugin, written in Golang. However it can be easily implemented as a separate containerd plugin that can asynchronously monitor the Available Pool usage and refill it with new devices. The device creation will be running in a different thread from the Deverlay Snapshotter and it would be easier to optimize the re-initialization policies to be invoked in idle time slots, solidating that the Deverlay Snapshotter remains highly available.

3.4.2 Bind mount or device mount

The Provisioner could also further optimize the Prepare method by keeping the devices, from the Available pool, mounted on a host path. During the Prepare method this path can be bind mounted instead of mounting the device itself; bind mounting is usually cheaper than device mount. These optimizations have not been evaluated yet as their added value in the case of microVMs would be possibly dominated by the sandbox initialization overhead. However the Deverlay Snapshotter can be easily integrated with Unikernel Runtimes like *runnc*. Prior works [**?**] have pointed out the inefficiency of container storage drivers for unikernels. We show that Deverlay scales Kata Containers running in lighweight Firecracker VMs, but we expect that spawning Unikernels with projects like Nabla Containers will further exploit Deverlay capabilities and unikernel - containers fast and elastic deployment.

3.5 Container storage for Unikernels

This section identifies the storage requirements of container-unikernels in the serverless computing context and presents a design and a prototype implementation of an end-to-end solution that address the needs of modern environments.

3.5.1 Enabling container storage for Unikernel Containers

It is common to describe the unikernel as an all-in-one binary. Although this can be true - one can bake in a unikernel binary everything that an application needs to run - it is certainly not optimal in a microservice environment where spawn time, reusability and ease of deployment are of great importance.

3.5.2 Storage classes for Unikernel Containers

To design a solution for container-unikernel storage handling, we first describe the types of storage access occuring in a typical microservice workflow. We decide on how to apply mechanisms employed in the Docker architecture, to efficiently handle storage access, based on the container ecosystem. We classify storage access of a unikernel container in four basic categories:

- **Application binary**: all the compiled code that an application needs in order to boot and run: monitor frontend (e.g Solo5), libOS (e.g rumprun), language runtimes (e.g python libraries). Essentially, this is the unikernel binary, a statically-linked minimal unit of compiled code inside the container, with size in the order of MBs. This forms the base layer for the container.

- **Library dependencies (modules)**: the equivalent of dynamic libraries for interpreted languages (e.g python, nodejs). They are user defined or common (e.g python ML) libraries and can be treated as independent standalone components. Their size can be multiple times bigger than the unikernel itself, in the order of 500MBs. Since these libraries are common, it is more than reasonable to be able to share them between different container images. To efficiently utilize available storage space and avoid data duplication, independent libraries should be treated as a different layers in the container - unikernel image stack.

- **Configuration data**: application specific or infrastructure related configuration data, such as network / distributed storage configuration and more. A user should be able to change the configuration without rebuilding the container - unikernel image. For example, a user should be able to change the configuration of a web service running in the container and at the same time be able to reuse the image. Since commonly the configuration data

size is relatively small, the user bind mounts functionality of Docker is a great fit for this use case. Thus, we argue that a container - unikernel should be able to support this.

- **I/O data**: In serverless computing there is no guarantee for the location of the executing function with respect to the I/O data it processes. In fact, it is common for I/O data to reside either in remote or distributed filesystems using library APIs to interface with user applications (i.e. RADOS/librados). We believe that I/O could be handled by porting the relevant runtime libraries or implementing split-driver solutions for container-unikernels.

3.5.3 Container storage for unikernels

In order to apply the above storage classification, while respecting the OCI runtime spec, we need to overcome the file-oriented approach of Docker layers, as well as the limitations of current Nabla containers implementations.

3.5.3.1 Extending the container image building process

The Docker storage driver implements two interfaces: (a) *ProtoDriver*, which defines the basic capabilities of a driver and (b) the *DiffDriver*, which is mainly responsible for the push/pull operations of Docker layers. The *Diff* (*ApplyDiff*) method is used to compress / decompress the files of a layer to a tar file when a docker push / pull operation is invoked.

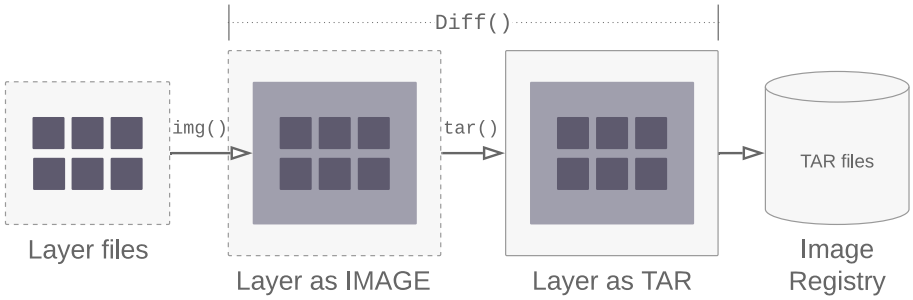


Figure 3.12: Build process of a container unikernel image

We introduce a container - unikernel storage driver, able to push / pull layers, each layer being represented as a simple image file, formatted in a common filesystem containing the layer's files. The `DiffDriver` creates these images and pushes them to the registry, as shown in Figure 3.12. The `ApplyDiff` method decompresses the tar archive which contains a single image file (e.g. ext2, iso) containing the files of the layer.

We outline the merits of our approach:

- In contrast to the current Nabla implementation, which creates at each run an iso image containing all the layers, we facilitate sharing of the layers among different container -

unikernel images. We reduce the amount of storage required to run multiple containers, and, additionally, we enable memory sharing on the host.

- Identical layers in use by two or more containers on the same host lead to effectively shared memory pages in host's page cache.
- The container - unikernel spawn time is reduced. The pulled image is already compatible with the unikernel block requirements and there is no need for re-creating the root filesystem as a fat iso image file at every boot.
- We eliminate the overhead of multiple file copies at boot time.

3.5.3.2 Extend Nabla containers

To apply the aforementioned layer design to Nabla containers, we introduce a number of changes in the Nabla containers code base.

The *Solo5* tender program uses a single file descriptor that corresponds to the image file used as block device. We create a structure of file descriptors, one for each block device - one for each layer. Each block device gets a unique identifier used to match to a virtual block device inside Rumprun.

The rumpuser *hypercall interface* is a hypervisor specific implementation that allows rumprun to access host resources. We implement the routines needed in order to:

- create the virtual block devices inside Rumprun.
- map them to Solo5 block devices in a similar way that Xen is mapping physical storage to virtual disks [26].
- perform the actual I/O access on the host images using the Solo5 block read / write hypercalls. When the Rumprun unikernel boots, the layers - images are union - mounted to create its root file system.

Another limitation of the current *Nabla container runtime* implementation is the absence of the user bind mounts (or volumes) option that Docker offers. To tackle configuration data sharing between the host and the unikernel container, we emulate the Docker user bind mounts functionality by creating a new layer (image file) at boot time. This image contains a copy of the files of the host that would normally be served as a bind mount inside the container. Assuming that the size of the configuration data is smaller by some orders of magnitude compared to the whole container size, the overhead added by creating the configuration image at boot time remains remarkably smaller than before. Our approach enables the user bind mounts functionality, while it preserves the security benefits by not directly exposing the host filesystem semantics to

the unikernel. We should note that there is no real-time sharing of bind mounted files between the container and the host. We argue that in the case of serverless computing the data that need to be shared or persistently changed are offered using other types of storage (e.g network). In the generic case, one could combine the above solution with the approach described in [62].

We believe that this is the proper way to take advantage of the common interfaces and filesystems already presented in the libOS frameworks (such as Rumprun), in order to recreate the host functionality inside the unikernel. We introduce an end-to-end design implementation that brings unikernels inside containers on par with the unikernels concept.

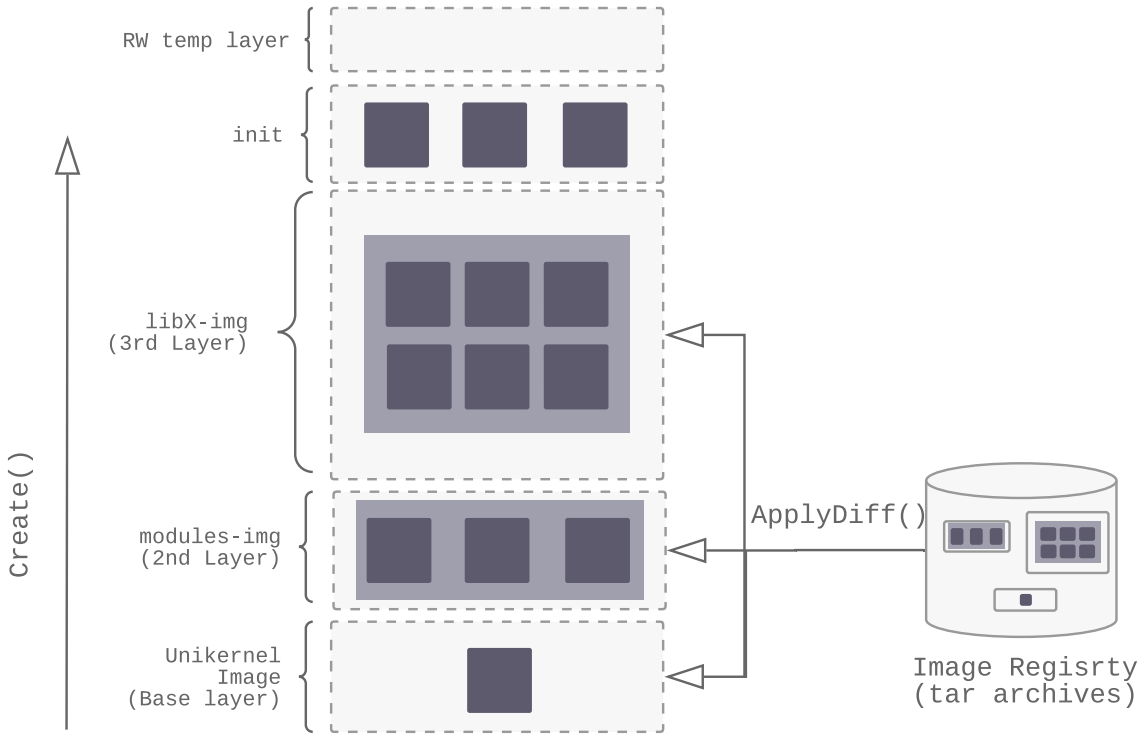


Figure 3.13: Create process of a container unikernel using filesystem images as individual storage layers.

Figure 3.13 [2] shows the create process for a 3 layer container - unikernel image running for the first time. The `Create` method creates an empty Base layer where the `ApplyDiff` uncompresses the unikernel binary tar. The 2nd and 3rd layers are created on top of the previous, with the addition (`ApplyDiff`) of the uncompressed `modules-img` and `libX-img` single images files formatted as a common filesystem containing the actual layer files. The image files (in our example `modules-img`, `libX-img`) are exposed to the unikernel as mountable block

[2]Image Registry contains the layer-images as tars. On the left, each thick box represents the uncompressed tar as layer-image. Arrows represent the Driver methods involved.

devices. The final view of the container - unikernel filesystem is created by the unikernel at boot time as the union mount of the above block devices and any (optional) user mounts/volumes.

3.5.4 Experimental results

We perform two simple experiments that showcase the benefits of our approach in terms of fast instantiation of a serverless function container, as well as the storage space re-use in the host. Specifically, we build a python unikernel that performs a simple HTTP request to a local

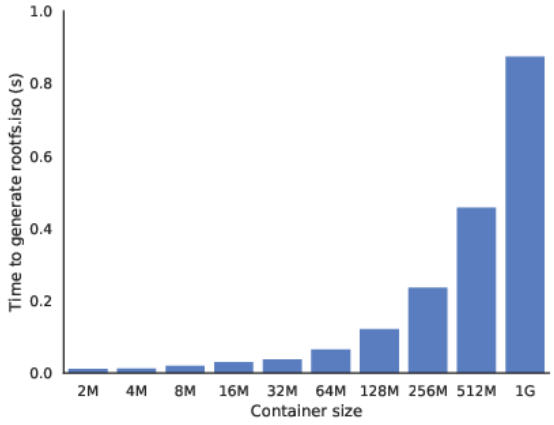


Figure 3.14: Time needed to generate the `rootfs.iso` as a function of the size of the container image

web server. We use Nabla containers to build the python unikernel and include the necessary libraries for python to handle the HTTP protocol (`requests`). When using the generic Nabla stack, we observe that for a 200MB container image (python binary + libraries), the amount of time needed to generate `rootfs.iso` is 337ms – this is almost 15% of the total execution time of the function (cold spawn to tear-down). To investigate the actual overhead of generating the image layers (`rootfs.iso`) at runtime, we plot the amount of time required as a function of the container size (Figure 3.14).

Since, in the serverless context, functions are usually short-lived (ranging from 500ms to a couple of seconds) this presents a significant overhead in spawn times.

We shift this operation to the build process, by injecting each image file (`rootfs-X.iso`) in its respective layer at the container image build time. As a result, we enable storage re-use, and, additionally, we remove the overhead of generating filesystem images at runtime.

To validate the hypothesis of storage re-use in the host, we spawn a number of identical containers simultaneously, and observe the total storage space used by the containers. Figure 3.15 plots the amount of storage space used as a function of the number of containers for the generic case (`nabla`), as well as our case (`nabla-ng`). We show that with our approach, there is 100%

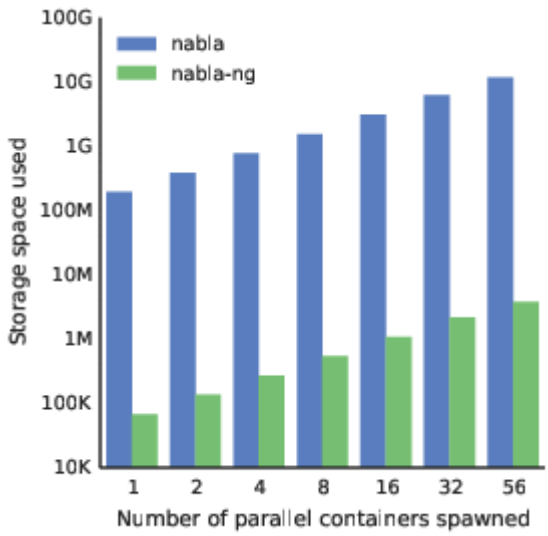


Figure 3.15: Storage space used as a function of the number of containers running on the system

reuse of the relevant image layers (`rootfs.iso`), as well as the unikernel binary in all cases. Specifically, spawning 56 container unikernels on a generic x86 machine[3] incurs less than 3MB of extra disk space, whereas spawning the same amount of containers using the generic Nabla stack incurs over 10GBs of extra disk space. This is expected as the `rootfs.iso` is being generated at runtime, so there is no way for the systems stack to know that the resulting file is actually the same. More importantly, all the accesses to the multiple rootfs files will be copied in the host page cache. Our approach potentiall decreases the total memory footprint to less than 3MiB while in the default nabla case multiple of GiB of duplicated date will reside in the host page cache.

[3]IBM cloud hosted Xeon(R) Gold 5120 CPU @ 2.20GHz

CHAPTER 4

Scaling functions in VMs with memory elasticity

This chapter presents Squeezy, a novel mechanism that extends the memory manager and the hot(un)plug interface of the OS in order to achieve rapid VM memory reclamation for serverless functions. Squeezy follows a special purpose design, tailored to resizing VMs running function instances under the multi-container-per-VM (§ 2.3.2) serverless model.

The first section of this chapter presents the motivation of this design by identifying an opportunity presented in serverless: the predictable memory requirements of the FaaS execution model, i.e., the fact that the memory limits for each function instance are defined by the user. This observation motivates the idea, described in detail in the next section 4.2, of segregating the memory allocations of functions that run concurrently inside a VM. This eliminates the interleaving of their footprints and hence enables the fast reclamation of their physical memory upon eviction. Implementation details and integration with a FaaS runtime are described in detail in the end of the section.

The experimental evaluation in Section 4.3 reveals that Squeezy is an order-of-magnitude faster than state-of-practice in reclaiming VM memory, typically allowing sub-second reclamation of multiple GiBs memory. It also achieves comparable P99 latency with statically overprovisioned VMs, while dynamically up- and down-scaling resources under realistic load [86].

4.1 Motivation

In § 2.3.3 we describe the pros and cons of the Serverless isolation models (§ 2.3). We consider the rigid resource over-provisioning one of the major drawbacks of the *multi-container-per-VM* (N:1) model (§ 2.3.2) that may overshadow its benefits. A natural way to address the problem is to reclaim fragmented memory and CPU resources at runtime. While CPU reclamation is well studied [33, 34, 65, 75, 101], in § 2.4 we discuss VM memory elasticity and elaborate on the challenges of resizing VM memory resources, especially within the tight latency limits of serverless [65]. Our conclusions are summarized as follows:

- Unplugging memory is dominated by page migrations.
- Zeroing out memory blocks further penalizes (un)plug operations.
- Memory allocation interleaves processes footprints within the same memory blocks, indirectly penalizing unplug performance.
- Page migrations can penalize the performance of other processes that run inside a VM which is being down-sized.

We observe that one of the reasons why (un)plugging suffers from long latencies is that it attempts to blindly reclaim guest OS memory. The hypervisor is unaware of the guest free memory blocks, due to the virtualization barrier, and thus scans the guest physical memory linearly, migrating occupied pages to isolate and remove memory blocks. While it is very hard to identify and isolate free memory on a general purpose VM, hosting multiple long-running workload(s) with working sets that vary over time and input, we observe that memory usage is far more predictable for the serverless use-case. The occupied and free memory inside a VM can be identified and managed at the granularity of the deployed function's memory resource limits, as defined by the user. *However, to be able to add and remove such coarse-grain blocks fast, avoiding migrations or free page searches, special mechanisms are necessary to guarantee their isolation.*

The goal of this design is to provide such mechanisms to dynamically adjust VM memory for the serverless use-case. It aims to reduce the memory fragmentation that severely penalizes the original N:1 model (§ 2.3.3). This mechanism is built on the existing memory hot(un)plugging mechanisms and attempts to address their performance pathologies, summarized above, for the specific use-case of Serverless.

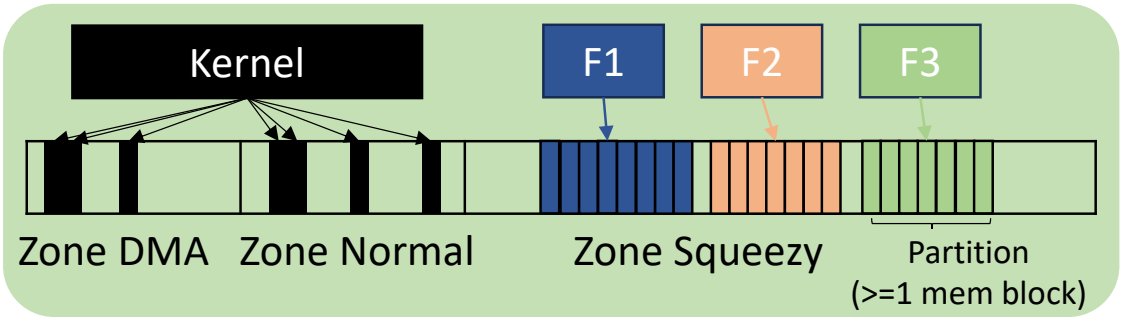


Figure 4.1: ZONE_SQUEEZY partitions and memory allocation

4.2 Memory Reclamation for Serverless functions

4.2.1 Overview

Squeezy is designed as an OS interface to efficiently manage the memory of serverless functions, enabling its rapid reclamation. Aligned to the analysis in Section 4.1, this design sets the following main goals:

Resource elasticity. Squeezy should be able to follow the memory requirements of the total number of alive function instances running inside the same VM, scaling up and down its memory resources accordingly. Squeezy should also be fast enough to cope with the bursty access pattern of serverless functions that frequently requires the up- and down-scaling of resources in sub-second intervals [54].

Resource sharing. Squeezy should allow the sharing of state, e.g., libraries, across the function instances that run in the same VM to preserve the memory saving benefits of the N:1 model.

Zero Interference. Squeezy should not affect the performance of already running function instances inside the VM.

To achieve these goals Squeezy introduces two mechanisms:

- It partitions guest memory across running function instances to isolate their footprints and prevent their interleaving.
- It makes the OS unplug interface (i.e., virtio-mem) aware of these partitions, to enable the fast and zero-interference reclamation of instances memory blocks as they terminate.

4.2.2 Memory Partitioning

Squeezy leverages the pre-defined memory requirements of function instances to be deployed in a VM under the N:1 model to partition the available guest memory into fixed-sized chunks. Each chunk can host a single instance. For this, Squeezy introduces a new memory zone in the OS memory manager (ZONE_SQUEEZY) that supports partitioning.

Squeezy memory cannot be used for generic allocations; it is reserved from the main OS allocation path. Instead, an instance has to explicitly ask to be backed by Squeezy resources, using a dedicated API described in § 4.2.5. In that case, it is mapped to a Squeezy partition and all its allocations (i.e., anonymous memory) are served from it. Once a partition is assigned to an instance, it is locked until the instance terminates. Figure 4.1 shows how different instances are isolated into different memory partitions under Squeezy.

A Squeezy partition is populated by a plug event, triggered by a serverless runtime (§ 4.2.6).

4.2.3 Distinguishing shared and private allocations

To preserve the memory savings of the N:1 model, Squeezy employs a shared partition per VM to back the file mappings of all instances. File mappings typically correspond to libraries and runtime dependencies shared across instances, i.e., they are instantiated once in memory and mapped multiple times. This partition is pre-populated at boot time for each VM based on the characteristics of each function. The anonymous mappings of the instances are backed by private partitions as discussed above.

4.2.4 Partition aware unplugging

As instances run isolated into Squeezy partitions, unplugging the memory of a terminated instance involves zero-migrations and thus zero-interference under Squeezy. Unplug operations are triggered by a serverless runtime (§ 4.2.6) when instance(s) inside a VM are recycled due to a drop in the incoming load. Squeezy extends the unplug interface of the OS to directly identify the Squeezy partitions that are emptied when the instance(s) terminate, and instantly offlines and removes the corresponding memory blocks without having to migrate any occupied pages.

4.2.5 Implementation details

We implement Squeezy in Linux kernel 6.6 extending: i) the memory manager and ii) the virtio-mem hot(un)plug guest driver.

Squeezy partitions. Squeezy partitions the guest physical memory into chunks that will host non-interleaved function instances. The size of each Squeezy partition must match the memory requirements of the function to be deployed in the specific VM under the N:1 serverless model. This parameter is known when the VM is set-up by a serverless runtime, as the memory resource limits of each function are defined by the user (§ 4.2). Thus we configure the partition size as a boot parameter for the guest OS.

We implement Squeezy partitions as different zones (*zone structs*) in the kernel, following the original ZONE_MOVABLE. Each such structure represents a guest physical memory region that populates a Squeezy partition (e.g., holding its starting address and size), is stored in the

per-NUMA node zonelists of the kernel (*node_zones*). Each partition is uniquely identified by its *partition id*. Squeezy currently supports only one memory node per VM. We consider NUMA topologies as future work (Section 5).

At boot time, we create N Squeezy zone structures, that initially link to empty partitions, thus setting the maximum concurrent functions that can be supported by each VM, i.e., only N instances can be concurrently deployed in the VM at any point in time. We use the term *concurrency factor* for N and expose it as a boot parameter to be set by the serverless runtime. Note that unlike an over-provisioned VM, this design pre-sets the maximum concurrency but does not pre-allocate the corresponding memory resources. The N Squeezy partitions are initially empty, at boot time; they are not backed by physical memory pages. They are instead populated and emptied dynamically by plug and unplug operations.

Squeezy populates only one partition at boot time, the *shared Squeezy partition*, that is dedicated to store shared runtime and language dependencies across concurrently running function instances in the VM. Its size is also a boot time parameter set by the serverless runtime.

Plugging a Squeezy partition. The Squeezy partition plugging starts when the hotplug driver inside the guest, i.e virtio-mem, receives a plug request from the hypervisor. The driver becomes aware that a specific range, corresponding to the partition(s) size, of its managed memory has been plugged by the hypervisor and uses the kernel memory onlining interfaces accordingly. We intercept the onlining process in order to instruct the kernel to correlate the memory range with the corresponding Squeezy partition(s), populate the free pages of the Squeezy partition and notify the users of the interface, as explained in the next paragraph. We note that if the system is configured to zero pages upon release (Section 2.4.1), we also modify the onlining path to skip zeroing Squeezy pages upon plugging, as they are already zeroed in the host.

The Squeezy interface. We design a system-call based interface for Squeezy, which allows the assignment of populated Squeezy partitions to function instances. The interface allows the calling process to request the OS to serve its memory allocations via Squeezy partitions. For each such request, we scan the list of Squeezy partitions (zonelist) in order to find an available (empty) Squeezy partition. Upon success, the partition is marked as reserved, increasing a reference counter, i.e. *partition_users*, and its partition id becomes associated with the calling process. Per-partition locks are used to avoid race conditions for concurrent requests.

Linux uses a memory descriptor (*mm_struct)* to represent each process's address space. We add a new field in the memory descriptor to store the partition id that is assigned to each Squeezy process. This is then used on the memory allocation path, in order to only allocate pages from the specific partition for the process.

As a Squeezy process starts executing inside the VM, its virtual memory accesses trigger the lazy allocation of guest physical memory through page fault traps. Squeezy intercepts the faults of Squeezy processes and implements a dedicated handler that allocates pages from the Squeezy

partition that is assigned to the process, instead of allocating pages from generic and shared OS zones (e.g. from ZONE_MOVABLE). This guarantees the confinement of the footprint of each Squeezy process within a single partition.

To preserve the sharing of language and runtime dependencies among the instances running in the VM, Squeezy distinguishes between faults for anonymous and file mappings. For the first, it allocates pages from the process-assigned Squeezy partition, as described above. For the latter, it uses the system-wide *shared Squeezy partition*, as described in previous paragraphs. It faults-in each file page once and subsequently maps it to the address space of any Squeezy process that touches it upon a fault. Essentially, this design implements a file caching layer for all function instances via a dedicated shared Squeezy partition.

If no available partitions exist, then the interface puts the requesting process in a *waitqueue*. Processes on the waitqueue are woken up upon two events: a) a plug operation that populates a Squeezy partition, and b) the release of a previously occupied Squeezy partition, i.e. when its *partition_users* variable drops to 0.

As discussed above, Squeezy applies the user-set memory limits of each function via the Squeezy partition size. If a function tries to allocate more memory than the size of the Squeezy partition, OS mechanisms (e.g, the OOM Killer) are triggered to kill the Squeezy process and prevent violations of partition isolation.

When a Squeezy process exits, its memory descriptor is destroyed. We intercept this path to also decrement the reference counter of its dedicated Squeezy partition (partition_users). When the counter drops to zero the partition, the interface kicks the *Squeezy waitqueue* described above to notify any waiting processes that a Squeezy partition is now available.

Handling forks. Every time a Squeezy process calls the clone system call to create a new thread or process, we assign the child to the parent's Squeezy partition, i.e., parent and children share the same partition. The intuition is that serverless functions by default do not call fork to scale [80, 89], i.e., they don't call fork to create new function instances. Thus we co-locate all threads and processes of a function instance on the same Squeezy partition. We also increment the partition reference counter (partition_users) as multiple memory descriptors now link to it.

Unplugging a Squeezy partition. The unplug operation starts upon receiving a request from the hypervisor. The hotplug driver, i.e virtio-mem, inside the guest, keeps track of free Squeezy partition(s) via their reference counter. When the driver receives an unplug operation from the hypervisor, it immediately offlines and removes a free partition without migrating any of the pages, as the partition is guaranteed to be empty. The serverless runtime coordinates the hypervisor unplug operations with the function recycle operations. It thus can know the exact number of greed Squeezy partitions in the VM. We also optimize the offlining path to exclude page zeroing by intercepting the generic allocation routines called during unplug as explained in Section 2.4.1, as pages will be zeroed by the host. The virtio-mem driver then informs the

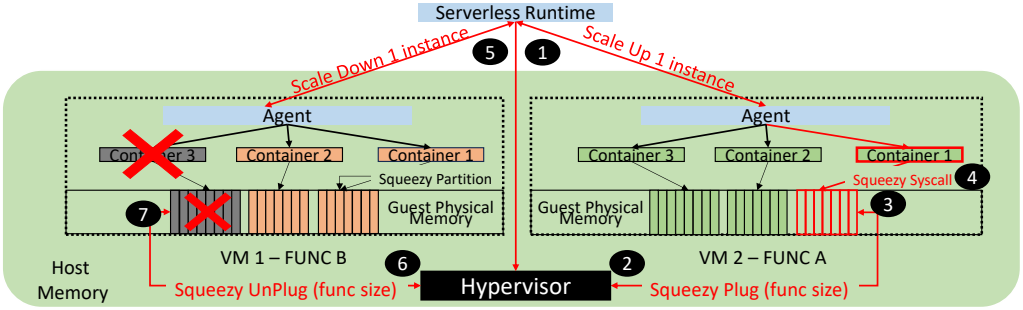


Figure 4.2: Squeezy integration to a serverless runtime

hypervisor that the memory blocks corresponding to the unplugged Squeezy partition(s) have been removed. The hypervisor immediately releases the pages to the host by using *madvise* to mark them MADV_DONTNEED.

4.2.6 Integration with a FaaS Runtime

Squeezy is an OS mechanism designed for the rapid reclamation of VM memory resources of serverless function instances when they terminate. A serverless runtime can leverage the Squeezy interface in order to (un)plug and (de)allocate Squeezy memory as it scales up and down function instances in the N:1 serverless model. Figure 4.2 shows how Squeezy can be integrated in such a runtime. For simplicity, we show the scaling up / down workflow for only one function instance. However, the same applies when the runtime concurrently scales up or down multiple function instances to respond to spikes in the incoming load. In Section 4.3, we evaluate a Squeezy-aware FaaS runtime based on Openwhisk.

Scaling Up. Figure 4.2 (right) shows the steps that take place when the runtime decides to spawn a new function instance to scale up resources for function A. The runtime sends a request to to the agent that runs inside the VM worker to scale up function instances (1). It also employs the hypervisor in the host to send a plug request to the guest, asking to add memory equal to the requirements of the instance, i.e., the memory size pre-defined by the user (2). The Squeezy plug path is triggered, and the guest populates a Squeezy partition with the added memory (3). The agent then spawns a new instance using the *Squeezy syscall interface* (4). A new container is created and starts executing with all its private allocations being served by and isolated in the just-plugged Squeezy partition.

Scaling Down. Figure 4.2 (left) shows the steps that take place when the runtime decides to evict a function instance to scale down resources for function B. The agent that runs inside the VM shuts down the function container and informs the runtime for the successful down-scaling (5). The runtime then employs the hypervisor in the host to send an unplug request to the guest

asking to reclaim memory equal to the memory freed by the recycle (6). The Squeezy fast unplug path is triggered in the guest, which i) identifies the empty Squeezy zone of the recycled container, ii) directly removes it without any migrations, and iii) notifies the hypervisor (7).

VM creation. Another event that must be adjusted when a serverless runtime adopts the Squeezy interface is VM creation. As discussed earlier when the runtime creates a VM that will host Squeezy function instances it must declare during boot i) the Squeezy private partition size, matching the memory resource limit set for the function, ii) the shared Squeezy partition size, matching the size of the runtime and language dependencies of the function, and iii) the maximum number of function instances that will be deployed in the VM (concurrency factor N).

4.3 Evaluation

4.3.1 Methodology

Hardware Setup

We evaluate Squeezy on a Intel server with two NUMA Nodes. Each node has 10 cores and 128GiB of memory each. We disable SMT in order to reduce variance in the results. The dynamically adjusted VMs, running the workloads, are configured with 10 cores and pinned to the same node (CPUs and memory). The initial VM memory size varies depending on the maximum hotplug memory, so that it can accommodate the required metadata (page structs) for the maximum hotplugged size ($inital_memory = max_memory * page_struct_size/page_size$). The maximum hotplug memory is always tied to the workload requirements and maximum concurrency and remains the same among experiments that compare the different mechanisms.

Software Components

We choose the Cloud Hypervisor (v38.0) as the VMM which includes a well tested rust implementation of the virtio-mem [50] hotplug mechanism. We build Squeezy on Linux Kernel 6.6.30.

Function	Description	Assigned vCPUs	Assigned Memory (MiB)
Cnn	JPEG classification CNN	0.5	384
Bert	BERT-based ML inference	1.0	640
BFS	Breadth-first search	0.5	384
HTML	HTML web service	0.2	384

Table 4.1: Squeezy evaluation workloads.

Workloads

Table 4.1 summarizes the serverless workloads used for evaluating Squeezy. We use the image classification benchmark (*Cnn*) from FunctionBench [58] and three representative real-world serverless workloads, also used in previous works [96]: the BERT-based ML inference [40] (*Bert*), a graph breadth-first-search workload (*BFS*) and an HTML web server (*HTML*). Based on previous works [96], we first experiment with the function resource limits on our setup until we achieve an acceptable response latency based on typical FaaS SLOs. We then assign different vCPU and memory limits to each workload, as shown in Table 4.1.

Metrics

Unplug latency: in all our experiments, the duration of the unplug request is obtained in the hypervisor and represents the time elapsed for an unplug request received by the hypervisor until the latter marks the reclaimed VM memory as MADV_DONTNEED, in order to immediately

release the requested size to the host. We obtain this metric from the Cloud Hypervisor tracing framework by slightly changing a few lines of code.

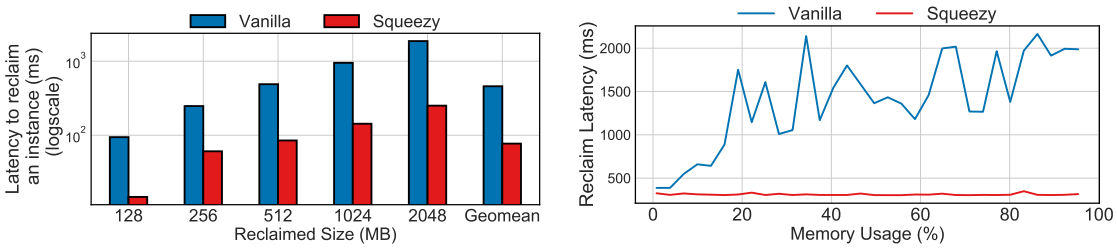
Function Latency: we report the tail latency (P99) of serverless functions, to showcase the effect of vanilla and Squeezy (un)plugging on function execution. We also report the per-second average of function end-to-end latency, in order to more accurately highlight the unplugging mechanism impact on function performance during shrinking events.

CPU Usage: we measure the CPU usage of the unplugging mechanism. We first enforce the guest kernel driver to always run on a specific vCPU, isolated from the micro benchmark process, by setting the virtio-mem interrupt affinity inside the guest. The corresponding vCPU thread is pinned to a specific physical core and additionally placed inside a threaded cgroup on the host. The same applies for the corresponding VMM virtio-mem thread on the host. We report the vCPU usage of the unplugging mechanism as measured by the CPU Accounting cgroup controller.

Evaluation scenarios

Microbenchmarks: we first evaluate the Squeezy mechanism unplug speed and CPU usage using the memhog microbenchmark [19]. Memhog repeatedly allocates and deallocates a specified amount of memory and as a side-effect it also stresses the CPU and memory of the host. We spawn multiple memhog processes on Squeezy partitions (squeezy) and on ZONE_MOVABLE (vanilla) and after a while we demand some of the memory back. The reclaim step involves freeing the memory by stopping memhog and immediately ask the hypervisor to unplug. We choose sizes that are representative of typical serverless memory limits in order to evaluate the unplugging speed of different Squeezy partitions.

Serverless functions: we evaluate 4 functions from FunctionBench [58] and [96], presented in Table 4.1. We deploy every function inside a dedicated VM, following the multi-container-per-VM model. We use Azure traces with bursty request patterns [86] to drive the invocations of every function and rely on a FaaS runtime to scale up and down function instances based on the incoming load (described next). We study the performance of three different configurations: a) using Squeezy, using Squeezy-aware virtio-mem, to scale up and down the VM resources along with the instances, b) using Vanilla with vanilla virtio-mem for the same purpose, and c) running the functions inside statically over-provisioned VMs (fixed maximum memory size throughout their lifecycle). We report the throughput (MiB/sec) with which Squeezy (a) and Vanilla (b) reclaims memory for every function. We additionally report the tail latency of successful function invocations for all configurations. Finally we study the impact of VM memory shrinking events (unplug) on the performance of other instances concurrently running inside the guest.



(a) Average latency (ms) to reclaim memory of different sizes from a guest under CPU load.

(b) Reclaiming 2GiB out of 64GiB of hotplugged memory as the overall memory usage increases.

Figure 4.3: Unplug VM memory performance.

FaaS Runtime

We build upon an Openwhisk-based runtime [96], by adding the required functionality to spawn secure containers inside VMs and resize the VM memory on demand, based on the number of concurrent existing function instances. The serverless runtime is responsible of issuing incoming request to the VM. A dispatcher process (Agent) inside the VM handles the incoming requests as described in Section 4.2.6 and shown in Figure 4.2. The Agent maintains a pool of idle containers per function. When no idle container is available for an incoming request, Agent scales up the the number of function instances. After a function instance successfully handles an incoming request, its container is placed back to the idle pool. A recycling operation, on a regular basis, asks the Agent to scale down the number of instances corresponding to any containers that are idle for more than the maximum keep-alive time. The Agent shrinks the VM memory based on the number of destroyed containers. We evaluate the function memory reclamation throughput based on the shrinking events through the experiment lifetime.

In order to get more accurate results the Agent equally pins the function instances to vCPUs based on the pre-configured vCPU weight per workload as shown in Table 4.1. For experimenting reasons we also add the option to choose specific vCPUs per request invocation.

4.3.2 Microbenchmarks

We spawn multiple instances of the memhog program as described in Section 4.3.1, in order to allocate almost all the free memory inside the VM and then release different amounts to be reclaimed by the hypervisor. We experiment with different plugged sizes and reclaim request sizes.

4.3.2.1 Unplug Speed

We first measure the raw reclamation speed for Squeezy and Vanilla. Figure 4.3a shows the reclamation latency for different memory sizes. Squeezy reclamation is an order of magnitude

faster than Vanilla as it totally avoids busy pages migration. We also note that as the requested size increases, the latency increases accordingly in both cases. This is related to the fact that Linux (un)plugs memory in the granularity of memory blocks (128MiB), as explained in § 2.4.1. The virtio-mem driver releases the memory in the same granularity. As a result, as the requested memory size increases, the operations invoked are multiplied in respect to the number of released memory blocks. We consider optimizing virtio-mem and Linux to handle unplug requests at larger granularities as future work (Section 5).

The performance of Vanilla is significantly affected by the number of free pages available inside the guest. We experiment on a VM with 64GB of plugged memory and repeatedly request to reclaim 2GiB out of 64GiB, while the guest memory usage increases. Figure 4.3b shows the unplug latency. We conclude the following:

- Squeezy has predictable and stable performance which is decoupled from the guest load. This stems from completely eliminating page migrations from the unplug path by making the hotplug driver aware of free (empty) Squeezy partitions.

- Vanilla latency exhibits an upward trend along with the memory usage, as the number of potentially busy pages per memory block increases, resulting in expensive migrations during unplug operations. Vanilla performance is unpredictable, i.e., there are fluctuations in performance, as the memhog benchmark inside the guest allocates pages randomly while vanilla virtio-mem tries to unplug memory blocks [50]. Squeezy instead instructs the virtio-mem driver to quickly aim for the unused memory blocks of an empty Squeezy partition providing stable performance.

4.3.2.2 CPU Usage during unplug

We also study the CPU usage of the two mechanisms, following the methodology in § 4.3.1. Figure 4.4 shows the cumulative CPU usage measured when we resize a VM with initially 16GiB of memory down to 512MiB in 32 reclamation steps, i.e., 32 unplug operations of 512MB each. We notice that Squeezy only slightly uses the vCPU throughout the experiment, while the vanilla mechanism keeps the CPU occupied in every step. Vanilla's CPU utilization is attributed to the page migrations that take place during unplug operations and as we show later can penalize performance of other processes concurrently running inside the VM. The step-wise increase represents the idle periods between the reclamation steps. The line for Squeezy is shorter than vanilla, as it finishes the unplugging much faster (§ 4.3.2.1).

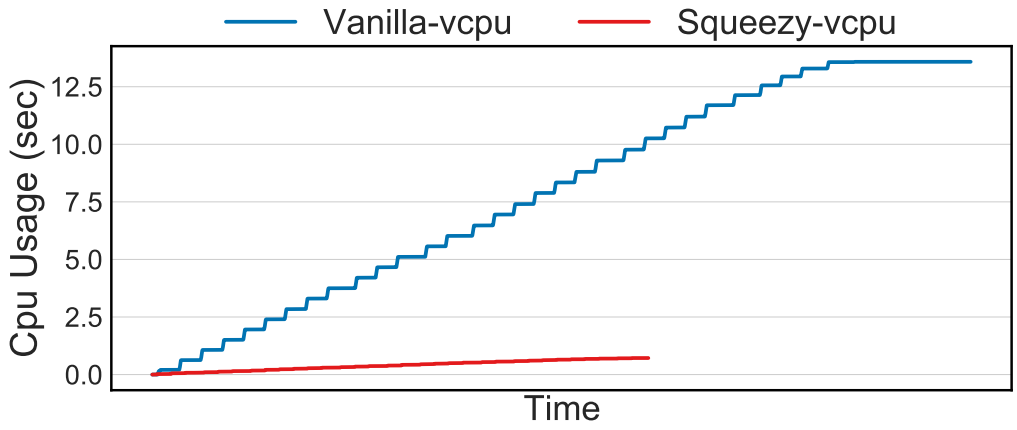


Figure 4.4: Cumulative CPU utilization for unpluging

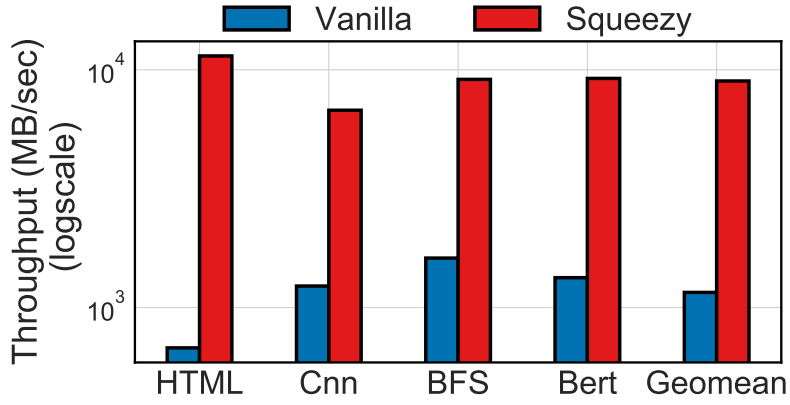


Figure 4.5: Throughput (MiB/sec) of reclaiming function instances memory based on a realistic FaaS trace experiment.

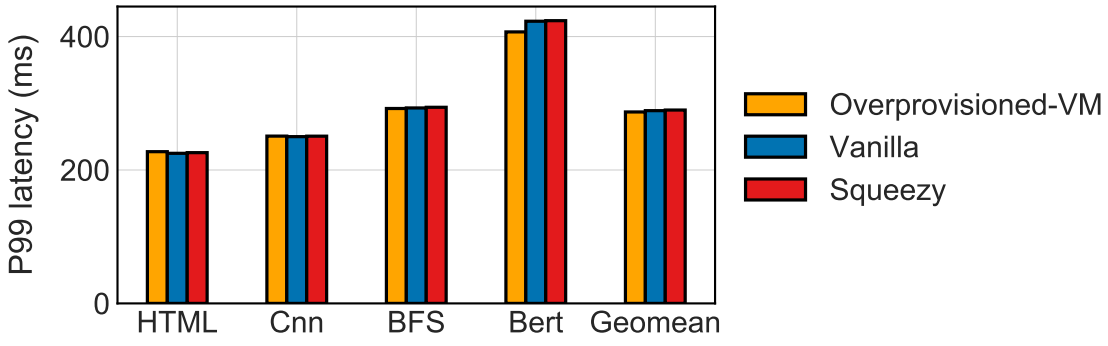


Figure 4.6: P99 latency of functions invocations.

4.3.3 Serverless workloads with real traces

4.3.3.1 Memory Reclamation

We select 4 Azure traces, map them randomly to our functions and use them to drive their load. Each function is placed in a separate VM, following the multiple-container-single-VM model. The maximum concurrency for each function is configured based on the VM vCPU count (10) divided by the *Assigned vCPUs* according to Table 4.1. It is passed as a boot parameter to its corresponding Squeezy VMs along with the function's memory requirements (§ 4.2.5). Squeezy uses this information to build its partitions.

For this experiment, the FaaS runtime (§ 4.3.1) dynamically scales up and down the number of instances per function, within the corresponding VMs, to respond to the incoming load. The scale up is combined with plug operations to size up the VM memory to allow hosting the new instances. Symmetrically, the scale down is combined with unplug operations that size down the VM to release the memory that was freed by recycling instances.

At the beginning of the traces used, there are bursts of requests that trigger the creation of multiple function instances (cold starts) and thus multiple plug events. Later the load drops abruptly, and the runtime starts to evict idle function instances. Figure 4.5 reports the throughput that each interface manages to reclaim memory during down-scaling. We observe that Squeezy achieves 7x higher throughput.

Figure 4.6 also reports the tail latency (99th percentile) of the requests per function for the same trace and for all interfaces. We also report the tail latency measured when we run the same experiment but using over-provisioned VMs, i.e., configured to have the maximum memory required throughout the experiment. These results study the impact of plug latency, which can penalize cold invocations. First we observe that vanilla and Squeezy have comparable performance, as expected, since Squeezy builds on top of the already optimized virtio-mem interface. More interestingly, we observe that dynamically hot(un)plugging memory while scaling up and down instances, using any interface, has comparable performance to using statically over-provisioned VMs. *The plug operations do not increase the tail latency.* Only Bert performance is slightly affected, for which we measure that plug operations take ≈30ms. *These results show that elasticity does not necessarily penalize performance.*

4.3.3.2 Interference during unplug

To investigate the impact of unplugging events on instances running on the same VM, we randomly choose two traces from the Azure Trace [86] as inputs for two of our workloads (Cnn and HTML) with similar memory requirements (same Squeezy partition size: 384MB) and *co-locate* them on the same VM. We instruct the runtime to pin the Cnn instances on 2 specific cores, one

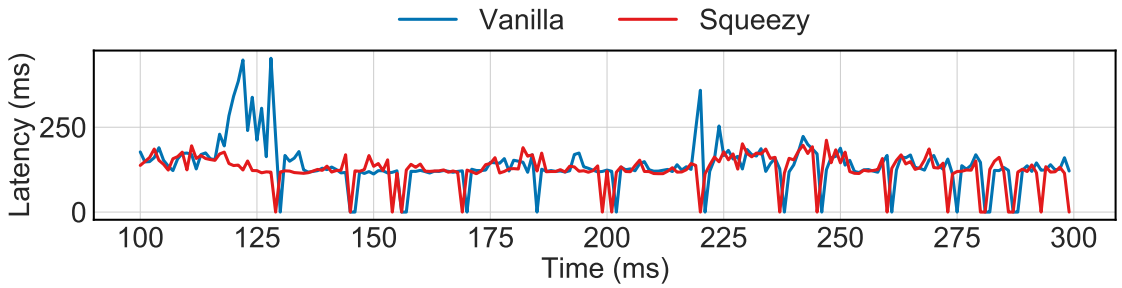


Figure 4.7: Average per second invocation end-to-end latency during memory shrinking events

of which also serves the virtio-mem interrupts inside the guest. The other workload is pinned to the other 8 vCPUs. The maximum number of instances for each function are adjusted according to the trace load and the vCPU assigned from Table 4.1 (Cnn 4, HTML 40). The runtime spawns the maximum amount of containers in order to handle the load. The idle lifetime is configured to 120 seconds. Figure 4.7 shows the average end-to-end latency per second for the Cnn requests. Vanilla shows a significant spike at around ≈125s of the experiment when the latency shows an increase of more than 100%. This is when the first shrinking event happens, after the runtime decides to evict 17 of the total spawned HTML function instances that were spawned more than 120 seconds ago in order to serve the initial burst of incoming requests. During this period the unplugging driver inside the guest has been scheduled to migrate pages using one of the vCPUs that are also assigned to the Cnn instances. The CPU utilization caused by the migrations (§ 4.3.2.2) penalizes the performance of Cnn function instances. A similar event occurs around the 225s of the experiment. On the other hand, the Cnn instances on the Squeezy VM configuration experience no similar impact during the scale down of HTML instances due to the zero-migrations design of Squeezy. We omit the HTML results as they exhibit the same performance for both configurations. The HTML function instances are pinned to a different subset of vCPUs, which don't include the vCPU that handles the resize requests. As a result, they experience no CPU interference during shrinking events.

We conclude that vanilla hot(un)plugging causes high CPU utilization during shrinking events due to page migration and we show how the workloads running inside the VM are affected by it. Squeezy overcomes this interference by eliminating the CPU cycles needed for migrations. Squeezy supports fast and efficient VM shrinking for serverless workloads and overcomes the need for redirecting the load to other VM workers during unplugging events, as happens in other related works (Section 6.3).

CHAPTER 5

Concluding the Thesis

5.1 VM Memory Elasticity

5.1.1 Reclamation chunk size

At the moment the maximum reclamation chunk size of Squeezy equal the size of a memory block in Linux, i.e. 128MiB. According to the evaluation results in § 4.3.2.1, as the reclamation memory size increases, the operations invoked are also increased leading to multiple *vmexits* in order to synchronize the communication between the host and the guest. Thus, I believe that the current reclamation chunk size limits to the performance potential of Squeezy. To that end, for the conclusion of my dissertation, I intend to support larger reclamation chunks up to the unplug request size. The intuition here is that the unplug request happens after instance(s) eviction and thus Squeezy partitioning guarantees that the unplugging mechanism can quickly identify and batch all the free memory blocks in one single operation.

5.1.2 Evaluation

To enhance the experimental evaluation of Squeezy I intend to include a series of experiments described in the next paragraphs.

5.1.2.1 End to end experiment

In order to show the memory efficiency of rapid reclamation for a more real-world scenario, I plan to include an end-to-end experiment where the microVMs involved are using a common memory pool with a limited size, i.e. plug requests are served only if the pool has enough free memory and unplug requests are releasing memory back to the pool. As a result the cold start time of a new instance inside a microVM will be related to the system's reclamation performance. I expect the Squeezy will outperform other mechanisms and improve the tai latency.

5.1.2.2 Comparison with VM ballooning and related work

The evaluation currently compares Squeezy with the state-of-the-art virtio-mem mechanism. However the ballooning approach - although it is slower - is still used by state-of-the-art hypervisors such as Firecracker. I intend to include an additional comparison with virtio-balloon.

Moreover related research is based on optimizing the ballooning mechanism using several techniques (§ 6.3). I intend to include a comparison with some of these techniques, e.g. larger balloon reclamation chunks and their impact on the system performance, e.g. new instance overhead.

5.1.2.3 Comparison with single instance model

In § 2.3.3 I argue that the single-container-per-vm model leads to cold start and memory overheads compared to the model that this thesis embraces. I intend to include an experiment that verifies this allegation.

5.1.2.4 Memory utilization

In order to better back the motivation behind VM memory elasticity for serverless I intend to include an experiment which shows that the total memory usage of the function instances inside a microVM is not aligned with the physical memory allocated size on the host. The reason for this mismatch is the virtualization gap, i.e. the fact that the host OS is not aware of free memory inside the guest.

5.1.2.5 Memory zeroing overheads

Finally I intend to include a study of the overhead stemming from zeroing pages during (un)plug operations. The results will include a breakdown of the unplug latency related to page migrations and page zeroing.

5.2 Future Work

5.2.1 Scaling Functions with VM snapshots

Single-container-per-VM (1:1) model (Horizontal). Section 2.3.3 describes how the 1:1 model penalizes cold start latency and leads to redundant memory resources due to the inherently duplicated runtime and OS/hypervisor state across microVMs. The state-of-the-art mitigation technique for these overheads is snapshotting [36, 90]. The method saves the state of a running microVM to storage, i.e. after the function instance has been initialized, and restores new instances execution from it. This reduces cold start latency, as restoration avoids boot and function initialization costs. It also reduces memory consumption as multiple instances can be restored from the same snapshot file, mapping it privately and thus sharing read-only state.

However, cold execution over snapshots, albeit faster than the baseline, is still significantly slower to cloning native processes (discussed next). Moreover, snapshotting has challenges at scale due to huge storage requirements, i.e. to store the snapshots for the tens of thousands of different functions/users [86].

N:1 model (Vertical). In the N:1 model a new instance is placed in an already running VM and as described in Section 2.3.3 the cold execution does not include the cost of VM booting. Moreover, memory consumption is decreased compared to the baseline 1:1 model due to state de-duplication. However, creating a new instance in this model still pays, by default, the function initialization overheads which dominate cold execution [41] and in terms of memory consumption there is substantial read-only function state still duplicated across instances [89]. The state-of-the-art mitigation technique [80, 89] for these overheads is to spawn new instances by *forking* an already initialized instance that runs inside the same VM. Apart from eliminating function initialization costs, fork semantics also enable the seamless sharing of read-only data across parent and child instances.

On the other hand, the background Section 2.3.3 identifies the significant drawback of this model: over-provisioned VMs are required, in terms of memory resources, to be able to host the maximum function concurrency (N). This potentially leads to idling allocated resources when there is no longer the need for N concurrent instances to serve the incoming load. To address this issue, this thesis and related works [45] propose VM resizing optimized with several techniques in order to mitigate memory reclamation overheads. However, even with agile VM resizing the N:1 modeling is based on VMs that either a) are very large and thus difficult to place/map to hardware resources, e.g. compute nodes, without leading to memory stranding [64] or b) change size over time, again complicating their scheduling/placement [42] [73].

We study the cold start performance and the memory consumption of the two models. We profile a new instance that a) is restored from a microVM snapshot file residing in the host's

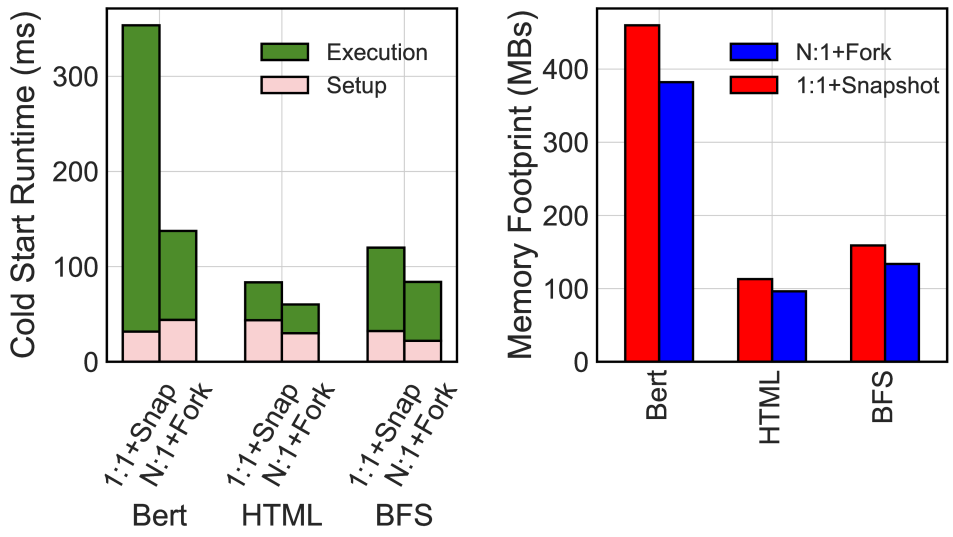


Figure 5.1: Comparison of the 1:1 and N:1 optimized models.

memory (1:1) and b) is forked from a running instance inside a microVM (N:1). Figure 5.1 summarizes our results. We observe that the 1:1 model's cold invocation, despite optimized with snapshotting, is 1.56-2.82x slower than the latter. We consider the nested faults required to restore OS, runtime and function state from the snapshot as the main source of overhead. We also observe that the N:1 model optimized with fork-based scaling, apart from faster, reduces also the memory footprint by 15%. Based on these results and the above discussion, we conclude that none of the models is always the perfect fit even when optimized. The 1:1 model offers a fine grained mapping of function instances to physical resources while the N:1 model can achieve better performance. We envision a new hybrid model that combines the benefits of both. To this direction, we identify open challenges in three areas:

Optimizing the N:1 model. Fork-based instance scaling has been mainly studied only for native containers [31, 80, 89]. We consider its extension for microVMs a necessary step for a high-performance hybrid scaling scheme. The fork model shares a substantial part of the function state between multiple instances, thus presents the opportunity to host multiple function workers inside a single microVM. For example, for all the workloads of Figure 5.1 we measure that a forked instance adds less than 30MiB of private (CoWed) anonymous memory. As users tend to request more memory than actually used by the function [53] we expect that the initial microVM will have idle memory enough to fork a number of extra instances under load.

Going hybrid. While forking N new instances within a (N:1) microVM offers better cold start performance and lower memory utilization compared to restoring N 1:1 instances from the same VM snapshot (Figure 5.1), it still has its own limits. The max concurrency factor (N) depends on the microVM memory size. However, we expect that larger memory configurations will fall back

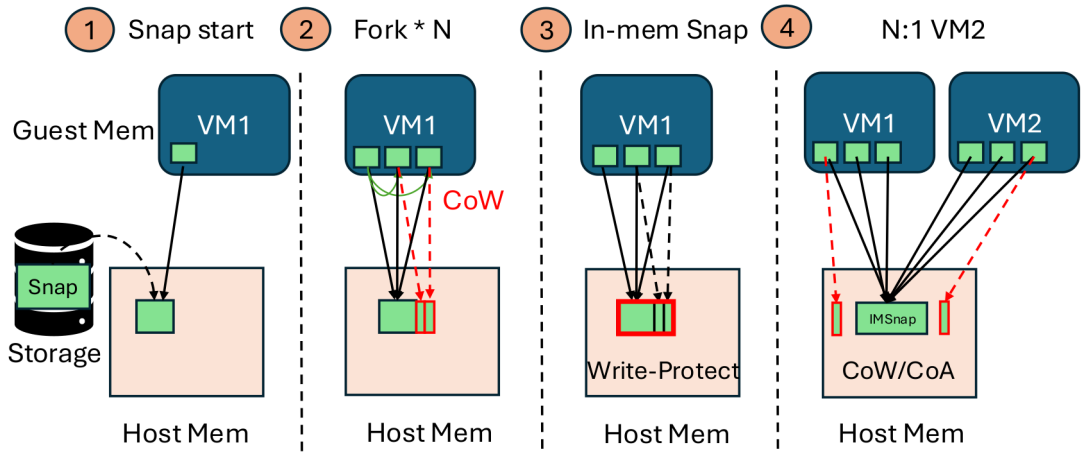


Figure 5.2: High level overview of the proposed hybrid model.

to the drawbacks of over-provisioned VMs. Moreover, we believe that the concurrency factor (N) is also limited by the CPU resources available in the sockets of the compute nodes: forking new instances within the same microVM and scheduling them to different NUMA nodes will penalize performance [45]. We consider creating a new microVM a better strategy in the above cases. We also observe that if we could checkpoint and restore efficiently a N:1 microVM, within the same or across sockets, we could scale faster the concurrency factor, i.e. restoring an N:1 microVM vs restoring a 1:1 microVM or forking a single instance. There are no existing techniques to provide such hybrid scaling efficiently.

Transforming the 1:1 model. We observe that the state-of-the-art mainly optimizes the 1:1 model by improving the IO performance of snapshot restoration while assuming that a snapshot per function is always available [36, 90]. We consider this impractical in terms of storage footprint, especially if we consider multiple snapshots per function to capture various concurrency factors, i.e. checkpoint scaled-up (N:1) microVMs. We envision a more practical solution where the base snapshot for each new function instance is a generic VM snapshot that shortcuts the microVM boot time and the FaaS runtime initialization and can be used for multiple different functions. This strategy is inspired by Zygotes, as developed and studied for native containers [80]. We also envision a new snapshotting scheme that allows to checkpoint-and-restore in-memory a scaled up N:1 VM, i.e. a VM with forked instances, on the fly to implement the fast up-scaling and socket crossing as described above.

5.2.1.1 Proposal Design Overview

In this section we briefly describe the high level idea of the hybrid model scaling. Its workflow is shown in Figure 5.2. The first cold invocation of a function (①) is served by restoring a microVM from a base generic snapshot that can reside either on storage or in-memory, cached

by a prior microVM. We anticipate the second to be more frequent as multiple functions share the same base snapshot. This first cold invocation pays the overhead of function state initialization. Subsequent instances are spawned by forking the existing instance inside the microVM until max concurrency is reached ((2)). When there is need for fast concurrency up-scaling, i.e. due to a high burst of incoming requests, and / or the resource limits of the microVM are reached, then in-memory snapshot of the running microVM is registered ((3)). Then a new microVM (VM2) - containing the same number of instances as the scaled up VM1 - is cloned / restored from this snapshot ((4)).

Generic storage snapshot. We aim to use generic base (”Zygote”) snapshots to restore multiple different functions. The first function instance on a compute node is restored from such a ”Zygote snapshot” (e.g Python Lambda Runtime image).

Optimizing the N:1 model. Subsequent function instances are forked until the microVM reaches its limits. We also consider exploring hotplugging more memory to the initial microVM in order to increase its concurrency factor. While the incoming load drops and these instances are recycled, scaling down can be achieved by evicting the N:1 microVM - releasing the anonymous memory touched by forks - after (optionally) pro-actively replacing it with a fresh one (switching from ((2)) straight to ((1)) in Figure 5.2).

Going hybrid by transforming the 1:1 model. To clone fast a N:1 microVM aiming a) to double max concurrency (N->2*N) or b) to cross the boundaries of a socket, we consider using novel in-memory snapshotting. In the current state of practice the only way to clone on-demand a scaled up microVM requires to first commit changes to the original snapshot file (e.g firecracker ”Diff” snapshots) which introduces overheads in the order of seconds. We, instead, checkpoint in memory the state of the running N:1 microVM by write-protecting the VM’s guest memory. We introduce new mechanisms to associate this checkpoint with a memfd [**?**]. From that point on we use this in-memory checkpoint (and the memfd handler) to restore new N:1 microVM(s) and all parent and child microVMs rely on Copy-On-Write faults for private data.

When restoring to a remote socket (NUMA node), the instances of the new microVM will pay slower read accesses (NUMA overheads). One way to mitigate this problem would be to copy the in-memory snapshot to the remote NUMA node. We consider to explore lazy on-demand transfer with a Copy-on-Access (CoA) mechanism.

5.2.2 Multiple Guests page cache deduplication

While this thesis implements a step forward de-duplucation of cached data in serverless there is still room for further improvements. More specifically the function dependencies, residing on the physical disk, are commonly cached both on host page cache and the guest(s) page cache. As future work I plan to investigate Direct Access for files inside the guest (DAX feature) in order to

bypass caching of function dependencies inside the guest. I expect that enabling DAX in guest's filesystem will improve the total memory footprint as well as the performance by avoiding an unnecessary copy inside the guest. Moreover DAX can potentially improve memory efficiency when using snapshots, as a part of the guest memory will be split from the snapshot file to the storage mechanism (i.e. virtio-pmem) backing file. Data that would otherwise be cached in the guest page cache and serialized to the guest memory snapshot file will then be shared among multiple microVMs started from different snapshot files and backed with the same read-only filesystem.

5.2.3 Optimizing storage for functions in VMs by identifying I/O patterns

Section 3 describes a design of container storage for functions running in VMs, based on the virtio-block (§ 2.2.5.1) driver. In the future I plan to investigate other storage drivers solutions for Virtual Machines, e.g. virtio-pmem and virtio-fs. A study of the storage access of serverless functions using tracing tools, e.g. ebpf, can reveal the I/O patterns (number of read / writes and sizes). Then a performance evaluation, based on these patterns, between the existing VM storage drivers can help in identifying the suitable choice for serverless workloads.

CHAPTER 6

Related Work

6.1 Memory Elasticity

To realize memory elasticity on physical machines, Infiniswap [49] relies on memory disaggregation implemented via a swapping-based interface over RDMA fabrics. FluidMem [38] targets VMs instead and improves upon the swap-based approach by utilizing the Linux userfaultfd interface [18]. Both Infiniswap and Fluidmem target disaggregated RDMA-accessible memory. By contrast, Squeezy targets FaaS microVMs and, by providing reliable and fast memory reclamation, enables the coupling of memory hot(un)plugging with function invocation.

Memory ballooning [51, 69, 84, 85, 91, 103] has been the state-of-practice interface to implement VM memory elasticity. Memory ballooning implements a paravirtualized interface, using a guest balloon driver controlled by the hypervisor. The VM is booted with a target maximum memory size and the hypervisor triggers the driver to inflate or deflate the balloon and decrease or increase the VM's available memory accordingly. When the hypervisor needs to reclaim memory, the guest driver allocates guest physical pages and reports reclaimed memory to the hypervisor, which can then be reused. To increase guest memory, the driver releases previously reclaimed memory back to the guest OS, increasing the VM's available memory. Ballooning is known to suffer from long inflation latencies as it triggers guest OS allocations that delay reclamation, especially when guest memory is fragmented [50, 76, 85]. A recent work [46] that uses ballooning to harvest idle VM memory resources, employs a large set of optimizations to work

around the method's high latency, e.g., reserving idle memory buffers to fall back to use when inflation is late to secure free blocks of memory.

Swap-based approaches [35,100] have also been used to improve VM memory elasticity using local (hypervisor) resources (memory), usually synergistically with memory ballooning. Previously, transcendent memory [39, 71] also used a swap-based interface to enable seamless VM memory scaling, via the Frontswap mechanism [16].

6.2 Serverless Scaling

In order to meet the extreme scalability and elasticity demands of the FaaS serverless computing paradigm, various techniques have been proposed. One line of research focuses on accelerating container sandboxing for serverless functions [31,80,83,89]. By contrast, we focus on microVM-based sandboxing. Snowflock [63] proposes VM cloning for scaling cloud workloads. We instead target FaaS functions running on microVMs. [72] showcases how VM-based sandboxing can outperform container-based sandboxing via lean virtual machines. Recently, microVM snapshotting has been proposed [36, 41, 56, 90] in order to accelerate function cold-starts. Squeezy unlocks increased performance by using lean microVMs, which can rapidly scale to accommodate multiple concurrent function invocations, thus avoiding the microVM initialization tax, and enabling functions to share the warmed-up state of other concurrently running functions on the same microVM.

6.3 Resource Harvesting

Harvesting of unused resources has been studied both in the context of virtual machines [34, 45, 93, 98] and serverless computing [101]. While these works focus on harvesting idle compute resources, i.e., CPUs, others [44, 46, 77, 87, 99] also support harvesting idle memory resources. More specifically, [46] evaluates harvesting memory with serverless workloads by introducing a number of optimizations including the preservation of large buffers of idle memory, proactive reclamation, and larger reclamation chunks. These optimizations aim to minimize the shrinking event impact on allocating resources for higher priority workloads but are not minimizing page migrations during unplug. The incoming requests to function instances are redirected to other VM workers during shrinking events in order to avoid the performance penalty. Squeezy builds upon the concept of idle memory harvesting in the context of FaaS and eliminates the shrinking event impact by targeting free memory ranges to avoid the migration overhead. By providing fast and reliable VM memory reclamation, Squeezy minimizes memory stranding and enables the rapid reuse of released memory for subsequent function invocations.

6.4 Container storage

DADI [?] focuses on remote layer snapshots and optimizes image level operations, such as image pull, in order to reduce cold starts when running an image which is not already downloaded on the host node. By introducing changes in the Linux kernel DADI presents a new driver able to apply block - based diffs between layers as described DADI Image Specification. However, like Devicemapper, each Container gets its own virtual block device and as a result there is no support for page cache sharing betweeen different containers from the same layers.

Slacker [?] aims to enable fast container distribution by lazily pulling image data from a remote centralized registry backed by NFS providing CoW snapshots mechanisms. Its main objective is to achieve faster startup times when the required image is not already downloaded on the host. However besides the first run which invokes the optimized pull mechanism, any subsequent runs are similar to other drivers. Moreover even with introducing kernel optimizations in order to enable cache sharing the I/O performance is worse that the AUFS driver. Cider [?], similarly to Slacker, implements remote images using Ceph snapshots. However these approaches can not efficiently share layer caching among parallel containers.

HP Mapper's [?] main goal is to provide fine-grained CoW and high cache efficiency by implementing three main modules. The Address Mapper translates virtual block numbers from the container device into physical block numbers using a two-level mapping tree. The Interceptor sends redundant requests directly to the cache while the Cache Manager monitors and evicts unnecessary pages.

Container storage for Unikernels

Koller et al. [62] propose a storage interface suitable for Nabla containers, focusing on alleviating the semantic gap between the host and the guest, while maintaining secure data access. Their proposal, taking into account modern container implementations, comprises of a de-privileged filesystem with mediated access for the guest. The idea is based on a combination of two storage mechanisms: read-only data files on the host and a read-write block device. The resulting interface requires a host loader that memory-maps host files and prepares the necessary metadata as a copy-on-write filesystem – a modified log-structured filesystem (such as LFS) is suggested as a possible target for an actual implementation.

CHAPTER 7

Summary

This thesis argues that we need to rethink OS mechanisms and concepts of traditional general purposed systems to target Serverless systems relying on multiple - guest - address spaces. It proposes two separate mechanisms designed to leverage cache de-duplication targeting efficient resource usage and improved tail latencies in serverless systems.

The first mechanism achieve this goal by introducing a novel container storage driver compatible with Virtual Machines that efficiently shares function's dependencies among multiple microVMs and eliminates initialization overheads coming from volumes provisioning.

The second mechanism introduces memory elasticity in the guest OS and implements an efficient mechanism for (re)claiming chunks of physical memory for / from function instances in microVMs.

The evaluation results show that both mechanisms applied in different isolation scenarios for serverless systems are improving cold start times, memory usage and end-to-end latencies.

Bibliography

[1] Alibaba serverless application engine. https://www.aliyun.com/product/aliware/sae.

[2] Aws lambda. https://aws.amazon.com/lambda.

[3] Azure functions. https://azure.microsoft.com/en-us/services/functions/.

[4] Cloudflare workers. https://workers.cloudflare.com/.

[5] Container Runtime Interface in Kubernetes. Accessed: 2025-03-24.

[6] Containerd snapshots. Accessed: 2025-03-24.

[7] Docker reference documentation. https://docs.docker.com/reference/.

[8] Don't Panic: Kubernetes and Docker. Accessed: 2025-03-24.

[9] firecracker-containerd. Accessed: 2025-03-24.

[10] Free page reporting. https://docs.kernel.org/mm/free_page_reporting.html.

[11] Google serverless computing. https://cloud.google.com/serverless.

[12] gVisor. Accessed: 2025-03-24.

[13] Huawei cloud functions. https://developer.huawei.com/consumer/en/agconnect/cloud-function/.

[14] Introduction to grpc. Accessed: 2025-03-24.

[15] Kata Containers. Accessed: 2025-03-24.

[16] Linux frontswap.

[17] Linux Kernel device-mapper. Accessed: 2025-03-24.

[18] Linux userfaultfd.

[19] memhog(8) man page.

[20] Memory balloon driver.

[21] Open Infrastructure Foundation. Accessed: 2025-03-24.

[22] overlay filesystem. Accessed: 2025-03-24.

[23] vhost-free vsock device, Dan Horobeanu. Accessed: 2021-12-18.

[24] virtio-9p. Accessed: 2025-03-24.

[25] virtiofs design. Accessed: 2025-03-24.

[26] Xen VBD interface, 2013. Accessed: 2019-09-15.

[27] Extraterrestrial Filesystem, 2014. Accessed: 2019-09-15.

[28] runsc, 2015. Accessed: 2025-03-24.

[29] Solomon Hykes, Introducing runC: a lightweight universal container runtime, 2015, 2015. Accessed: 2025-03-24.

[30] Agache, A., Brooker, M., Iordache, A., Liguori, A., Neugebauer, R., Piwonka, P., and Popa, D.-M. Firecracker: Lightweight virtualization for serverless applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)* (Santa Clara, CA, Feb. 2020), USENIX Association, pp. 419–434.

[31] Akkus, I. E., Chen, R., Rimac, I., Stein, M., Satzke, K., Beck, A., Aditya, P., and Hilt, V. {SAND}: Towards {High-Performance} serverless computing. In *2018 Usenix Annual Technical Conference (USENIX ATC 18)* (2018).

[32] Alverti, C., Psomadakis, S., Karakostas, V., Gandhi, J., Nikas, K., Goumas, G., and Koziris, N. Enhancing and Exploiting Contiguity for Fast Memory Virtualization. In *Proceedings of the 47th ACM/IEEE Annual International Symposium on Computer Architecture* (2020).

[33] Amazon AWS. Amazon ec2 spot instances, 2022. https://aws.amazon.com/ec2/spot/[Online; accessed 23-October-2022].

[34] Ambati, P., Goiri, I., Frujeri, F., Gun, A., Wang, K., Dolan, B., Corell, B., Pasupuleti, S., Moscibroda, T., Elnikety, S., Fontoura, M., and Bianchini, R. Providing SLOs for Resource-Harvesting VMs in cloud platforms. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)* (Nov. 2020), USENIX Association, pp. 735–751.

[35] Amit, N., Tsafrir, D., and Schuster, A. Vswapper: A memory swapper for virtualized environments. *Acm Sigplan Notices 49*, 4 (2014).

[36] Ao, L., Porter, G., and Voelker, G. M. Faasnap: Faas made fast using snapshot-based vms. In *Proceedings of the Seventeenth European Conference on Computer Systems* (2022).

[37] Barham, P., Dragovic, B., Fraser, K., Hand, S., Harris, T., Ho, A., Neugebauer, R., Pratt, I., and Warfield, A. Xen and the art of virtualization. *SIGOPS Oper. Syst. Rev.* (2003).

[38] Caldwell, B., Goodarzy, S., Ha, S., Han, R., Keller, E., Rozner, E., and Im, Y. Fluidmem: Full, flexible, and fast memory disaggregation for the cloud. In *2020 IEEE 40th International Conference on Distributed Computing Systems (ICDCS)* (2020), IEEE.

[39] Dan Magenheimer . Transcendent memory in a nutshell, 2011.

[40] Devlin, J., Chang, M., Lee, K., and Toutanova, K. BERT: pre-training of deep bidirectional transformers for language understanding. *CoRR abs/1810.04805* (2018).

[41] Du, D., Yu, T., Xia, Y., Zang, B., Yan, G., Qin, C., Wu, Q., and Chen, H. Catalyzer: Sub-millisecond startup for serverless computing with initialization-less booting. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems* (2020).

[42] Ekici, A. Bin packing problem with conflicts and item fragmentation. *Computers & Operations Research 126* (2021), 105113.

[43] Fu, X., and Zhou, C. Predicted affinity based virtual machine placement in cloud computing environments. *IEEE Transactions on Cloud Computing 8*, 1 (2020), 246–255.

[44] Fuerst, A., Ali-Eldin, A., Shenoy, P., and Sharma, P. Cloud-scale vm-deflation for running interactive applications on transient servers. In *Proceedings of the 29th International Symposium on High-Performance Parallel and Distributed Computing* (2020).

[45] Fuerst, A., Novaković, S., Goiri, Í., Chaudhry, G. I., Sharma, P., Arya, K., Broas, K., Bak, E., Iyigun, M., and Bianchini, R. Memory-harvesting vms in cloud platforms. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (2022).

[46] Fuerst, A., Novaković, S., Goiri, I. n., Chaudhry, G. I., Sharma, P., Arya, K., Broas, K., Bak, E., Iyigun, M., and Bianchini, R. Memory-harvesting vms in cloud platforms. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (New York, NY, USA, 2022), ASPLOS '22, Association for Computing Machinery, p. 583–594.

[47] Fuerst, A., and Sharma, P. Faascache: keeping serverless computing alive with greedy-dual caching. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (2021), ASPLOS '21.

[48] Gandhi, J., Hill, M. D., and Swift, M. M. Agile Paging: Exceeding the Best of Nested and Shadow Paging. In *Proceedings of the 43rd ACM/IEEE International Symposium on Computer Architecture* (2016).

[49] Gu, J., Lee, Y., Zhang, Y., Chowdhury, M., and Shin, K. G. Efficient memory disaggregation with infiniswap. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)* (2017).

[50] Hildenbrand, D., and Schulz, M. virtio-mem: paravirtualized memory hot(un)plug. In *Proceedings of the 17th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments* (2021), VEE 2021.

[51] Hu, J., Bai, X., Sha, S., Luo, Y., Wang, X., and Wang, Z. Hub: hugepage ballooning in kernel-based virtual machines. In *Proceedings of the International Symposium on Memory Systems* (2018).

[52] Jonas, E. e. a. Cloud programming simplified: A berkeley view on serverless computing. Tech. Rep. UCB/EECS-2019-3, EECS Department, University of California, Berkeley, Feb 2019.

[53] Joosen, A., Hassan, A., Asenov, M., Singh, R., Darlow, L., Wang, J., and Barker, A. How does it function? characterizing long-term trends in production serverless workloads. In *Proceedings of the 2023 ACM Symposium on Cloud Computing* (2023).

[54] Joosen, A., Hassan, A., Asenov, M., Singh, R., Darlow, L., Wang, J., Deng, Q., and Barker, A. Serverless cold starts and where to find them, 2024.

[55] Kantee, A., and Cormack, J. Rump Kernels: No OS? No Problem! *;login: 39* (2014).

[56] Katsakioris, C., Alverti, C., Karakostas, V., Nikas, K., Goumas, G., and Koziris, N. Faas in the age of (sub-)ms i/o: A performance analysis of snapshotting. In *Proceedings of the 15th ACM International Conference on Systems and Storage* (New York, NY, USA, 2022), SYSTOR '22, Association for Computing Machinery, p. 13–25.

[57] Kim, D. K., and Roh, H.-G. Scheduling containers rather than functions for function-as-a-service. In *2021 IEEE/ACM 21st International Symposium on Cluster, Cloud and Internet Computing (CCGrid)* (2021).

[58] Kim, J., and Lee, K. Practical cloud workloads for serverless faas. In *Proceedings of the ACM Symposium on Cloud Computing* (2019).

[59] Kivity, A., Laor, D., Costa, G., Enberg, P., Har'El, N., Marti, D., and Zolotarov, V. Osv—optimizing the operating system for virtual machines. In *2014 USENIX Annual Technical Conference (USENIX ATC 14)* (Philadelphia, PA, 2014), USENIX Association, pp. 61–72.

[60] Kohli, S., Kharbanda, S., Bruno, R., Carreira, J., and Fonseca, P. Pronghorn: Effective Checkpoint Orchestration for Serverless Hot-Starts. In *Proceedings of the 19th European Conference on Computer Systems (EuroSys)* (2024).

[61] Koller, R., and Williams, D. Will serverless end the dominance of linux in the cloud? In *Proceedings of the 16th Workshop on Hot Topics in Operating Systems* (New York, NY, USA, 2017), HotOS '17, ACM, pp. 169–173.

[62] Koller, R., and Williams, D. An ounce of prevention is worth a pound of cure: Ahead-of-time preparation for safe high-level container interfaces. In *11th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 19)* (Renton, WA, July 2019), USENIX Association.

[63] Lagar-Cavilla, H. A., Whitney, J. A., Scannell, A. M., Patchin, P., Rumble, S. M., De Lara, E., Brudno, M., and Satyanarayanan, M. Snowflock: rapid virtual machine cloning for cloud computing. In *Proceedings of the 4th ACM European conference on Computer systems* (2009).

[64] Li, H., Berger, D. S., Hsu, L., Ernst, D., Zardoshti, P., Novakovic, S., Shah, M., Rajadnya, S., Lee, S., Agarwal, I., Hill, M. D., Fontoura, M., and Bianchini, R. Pond: Cxl-based memory pooling systems for cloud platforms. In *Proceedings of the 28th ACM*

International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (2023), ASPLOS 2023.

[65] Li, Z., Cheng, J., Chen, Q., Guan, E., Bian, Z., Tao, Y., Zha, B., Wang, Q., Han, W., and Guo, M. RunD: A lightweight secure container runtime for high-density deployment and high-concurrency startup in serverless computing. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)* (Carlsbad, CA, July 2022), USENIX Association, pp. 53–68. https://www.usenix.org/conference/atc22/presentation/li-zijun-rund.

[66] Li, Z., Guo, L., Cheng, J., Chen, Q., He, B., and Guo, M. The serverless computing survey: A technical primer for design architecture. *ACM Comput. Surv.* (2022).

[67] Lin, X., Lei, L., Wang, Y., Jing, J., Sun, K., and Zhou, Q. A measurement study on linux container security: Attacks and countermeasures. In *Proceedings of the 34th Annual Computer Security Applications Conference* (New York, NY, USA, 2018), ACSAC '18, ACM, pp. 418–429.

[68] Linux. Memory hot(un)plug, 2024. https://docs.kernel.org/admin-guide/mm/memory-hotplug.html[Online; accessed 8-Oct-2024].

[69] Liu, H., Jin, H., Liao, X., Deng, W., He, B., and Xu, C.-z. Hotplug or ballooning: A comparative study on dynamic memory management techniques for virtual machines. *IEEE Transactions on parallel and distributed systems 26*, 5 (2014), 1350–1363.

[70] Madhavapeddy, A., Mortier, R., Rotsos, C., Scott, D., Singh, B., Gazagnaire, T., Smith, S., Hand, S., and Crowcroft, J. Unikernels: Library operating systems for the cloud. *SIGPLAN Not. 48*, 4 (Mar. 2013), 461–472.

[71] Magenheimer, D., Mason, C., McCracken, D., and Hackel, K. Transcendent memory and linux. In *Proceedings of the Linux Symposium* (2009).

[72] Manco, F., Lupu, C., Schmidt, F., Mendes, J., Kuenzer, S., Sati, S., Yasukata, K., Raiciu, C., and Huici, F. My vm is lighter (and safer) than your container. In *Proceedings of the 26th Symposium on Operating Systems Principles* (2017).

[73] Mandal, C., Chakrabarti, P., and Ghose, S. Complexity of fragmentable object bin packing and an application. *Computers & Mathematics with Applications 35*, 11 (1998), 91–97.

[74] Microsoft Azure. Azure functions hosting options, 2022. https://learn.microsoft.com/en-us/azure/azure-functions/functions-scale[Online; accessed 23-October-2022].

[75] Microsoft Azure. Azure spot virtual machines, 2022. https://azure.microsoft.com/en-us/products/virtual-machines/spot/#overview[Online; accessed 23-October-2022].

[76] Moniruzzaman, A. B. M. Analysis of memory ballooning technique for dynamic memory management of virtual machines (vms), 2014.

[77] Mvondo, D., Bacou, M., Nguetchouang, K., Ngale, L., Pouget, S., Kouam, J., Lachaize, R., Hwang, J., Wood, T., Hagimont, D., et al. Ofc: an opportunistic caching system for faas platforms. In *Proceedings of the Sixteenth European Conference on Computer Systems* (2021).

[78] Nikolos, O. L., Goumas, G., and Koziris, N. Deverlay: Container snapshots for virtual machines. In *2022 22nd IEEE International Symposium on Cluster, Cloud and Internet Computing (CCGrid)* (2022), pp. 11–20.

[79] Nikolos, O. L., Papazafeiropoulos, K., Psomadakis, S., Nanos, A., and Koziris, N. Extending storage support for unikernel containers. In *Proceedings of the 5th International Workshop on Serverless Computing* (New York, NY, USA, 2019), WOSC '19, Association for Computing Machinery, p. 31–36.

[80] Oakes, E., Yang, L., Zhou, D., Houck, K., Harter, T., Arpaci-Dusseau, A., and Arpaci-Dusseau, R. {SOCK}: Rapid task provisioning with {Serverless-Optimized} containers. In *2018 USENIX annual technical conference (USENIX ATC 18)* (2018).

[81] Olivier, P., Chiba, D., Lankes, S., Min, C., and Ravindran, B. A Binary-compatible Unikernel. In *Proceedings of the 15th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments* (New York, NY, USA, 2019), VEE 2019, ACM, pp. 59–73.

[82] Panwar, A., Prasad, A., and Gopinath, K. Making huge pages actually useful. In *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems* (2018), ASPLOS '18.

[83] Roy, R. B., Patel, T., and Tiwari, D. Icebreaker: Warming serverless functions better with heterogeneity. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (2022).

[84] Salomie, T.-I., Alonso, G., Roscoe, T., and Elphinstone, K. Application level ballooning for efficient server consolidation. In *Proceedings of the 8th ACM European Conference on Computer Systems* (2013), pp. 337–350.

[85] Schopp, J. H., Fraser, K., and Silbermann, M. J. Resizing memory with balloons and hotplug. In *Proceedings of the Linux Symposium* (2006).

[86] Shahrad, M., Fonseca, R., Goiri, I., Chaudhry, G., Batum, P., Cooke, J., Laureano, E., Tresness, C., Russinovich, M., and Bianchini, R. Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider. In *2020 USENIX annual technical conference (USENIX ATC 20)* (2020).

[87] Sharma, P., Ali-Eldin, A., and Shenoy, P. Resource deflation: A new approach for transient resource reclamation. In *Proceedings of the Fourteenth EuroSys Conference 2019* (2019), EuroSys '19.

[88] Shin, W., Kim, W.-H., and Min, C. Fireworks: A Fast, Efficient, and Safe Serverless Framework using VM-level post-JIT Snapshot. In *Proceedings of the 17th European Conference on Computer Systems (EuroSys)* (2022).

[89] Stojkovic, J., Xu, T., Franke, H., and Torrellas, J. Mxfaas: Resource sharing in serverless environments for parallelism and efficiency. In *Proceedings of the 50th Annual International Symposium on Computer Architecture* (2023), ISCA '23.

[90] Ustiugov, D., Petrov, P., Kogias, M., Bugnion, E., and Grot, B. Benchmarking, analysis, and optimization of serverless function snapshots. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'21)* (2021), ACM.

[91] Waldspurger, C. A. Memory resource management in vmware esx server. *ACM SIGOPS Operating Systems Review 36*, SI (2002).

[92] Wang, L., Li, M., Zhang, Y., Ristenpart, T., and Swift, M. Peeking behind the curtains of serverless platforms. In *2018 USENIX annual technical conference (USENIX ATC 18)* (2018).

[93] Wang, Y., Arya, K., Kogias, M., Vanga, M., Bhandari, A., Yadwadkar, N. J., Sen, S., Elnikety, S., Kozyrakis, C., and Bianchini, R. Smartharvest: Harvesting idle cpus safely and efficiently in the cloud. In *Proceedings of the Sixteenth European Conference on Computer Systems* (2021).

[94] Williams, D., and Koller, R. Unikernel monitors: Extending minimalism outside of the box. In *Proceedings of the 8th USENIX Conference on Hot Topics in Cloud Computing* (Berkeley, CA, USA, 2016), HotCloud'16, USENIX Association, pp. 71–76.

[95] Williams, D., Koller, R., Lucina, M., and Prakash, N. Unikernels as processes. In *Proceedings of the ACM Symposium on Cloud Computing* (New York, NY, USA, 2018), SoCC '18, ACM, pp. 199–211.

[96] Xu, C., Liu, Y., Li, Z., Chen, Q., Zhao, H., Zeng, D., Peng, Q., Wu, X., Zhao, H., Fu, S., and Guo, M. Faasmem: Improving memory efficiency of serverless computing with memory pool architecture. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (2024), ASPLOS '24.

[97] Yan, Z., Lustig, D., Nellans, D., and Bhattacharjee, A. Translation Ranger: Operating System Support for Contiguity-aware TLBs. In *Proceedings of the 46th ACM/IEEE International Symposium on Computer Architecture* (2019).

[98] Yang, F., Wang, L., Xu, Z., Zhang, J., Li, L., Qiao, B., Couturier, C., Bansal, C., Ram, S., Qin, S., et al. Snape: Reliable and low-cost computing with mixture of spot and on-demand vms. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (2023).

[99] Yu, H., Fontenot, C., Wang, H., Li, J., Yuan, X., and Park, S.-J. Libra: Harvesting idle resources safely and timely in serverless clusters. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing* (2023).

[100] Zhang, Q., Liu, L., Su, G., and Iyengar, A. Memflex: A shared memory swapper for high performance vm execution. *IEEE transactions on Computers 66*, 9 (2017).

[101] Zhang, Y., Goiri, I. n., Chaudhry, G. I., Fonseca, R., Elnikety, S., Delimitrou, C., and Bianchini, R. *Faster and Cheaper Serverless Computing on Harvested Resources*. Association for Computing Machinery, New York, NY, USA, 2021, p. 724–739.

[102] Zhao, K., Xue, K., Wang, Z., Schatzberg, D., Yang, L., Manousis, A., Weiner, J., Van Riel, R., Sharma, B., Tang, C., and Skarlatos, D. Contiguitas: The Pursuit of Physical Memory Contiguity in Datacenters. In *Proceedings of the 50th ACM/IEEE Annual International Symposium on Computer Architecture* (2023).

[103] Zhao, W., Wang, Z., and Luo, Y. Dynamic memory balancing for virtual machines. *ACM SIGOPS Operating Systems Review 43*, 3 (2009).